

RĪGAS VALSTS TEHNIKUMS

DATORIKAS NODAĻA

Izglītības programma: Programmēšana

KVALIFIKĀCIJAS DARBS

“Ēdiena recepšu meklēšanas un ģenerēšanas tīmekļa lietotne”

Paskaidrojošais raksts 60 lpp.

Audzēknis:

Aleksis Kahanovskis DP4-1

Prakses vadītājs:

Monta Montvida

Nodaļas vadītājs:

Normunds Barbāns

Rīga, 2026

ANOTĀCIJA

Ēdiena recepšu meklēšanas un ģenerēšanas tīmekļa lietotne "FOODYML" ir izstrādāta, lai lietotāji varētu ērti meklēt, filtrēt un ģenerēt ēdiena receptes latviešu valodā, ņemot vērā individuālos uztura ierobežojumus un alerģijas. Sistēmas izstrādes laikā tika izmantotas tādas tehnoloģijas kā HTML5, CSS, JavaScript, PHP 8, Laravel, Vue 3, Inertia.js, Bootstrap 5 un MySQL. Recepšu ģenerēšanai tika izmantots DeepSeek mākslīgā intelekta API, bet PDF dokumentu veidošanai - DomPDF bibliotēka. Lietotāju autentifikācijai tika izmantots Laravel Sanctum un Google OAuth, bet resursu apkopošanai - Vite.

Kvalifikācijas darbs ietver ievadu, uzdevuma nostādni, prasību specifikāciju, uzdevuma risināšanas līdzekļu izvēles pamatojumu, programmatūras produkta modelēšanas un projektēšanas aprakstu, datu struktūru aprakstu un secinājumus. Ievadā ir aprakstīta recepšu meklēšanas sistēmas aktualitāte latviešu valodā. Uzdevuma nostādnē ir norādīti uzdevumi, kurus sistēmai bija nepieciešams veikt. Prasību specifikācija sastāv no ieejas un izejas informācijas, kā arī no sistēmas funkcionālajām un nefunkcionālajām prasībām. Uzdevuma risināšanas līdzekļu izvēles pamatojumā ir norādīti izstrādāšanai izmantotie līdzekļi un to pielietojums. Programmatūras produkta modelēšanas un projektēšanas apraksts ietver sistēmas struktūras modeli, datu plūsmu modeli un funkcionālo sistēmas modeli. Datu struktūru apraksē tiek parādīta datu bāzes relāciju shēma, kā arī tabulu struktūra ar datu tipu un datu garuma norādīšanu. Lietotāja ceļvedī ir norādītas sistēmas prasības aparatūrai un programmatūrai, instalācija un palaišana, kā arī programmas lietošanas pamācība. Testa piemērā ir dots detalizēts lietojamības testēšanas apraksts ar vizuāliem attēliem.

Kvalifikācijas darbs sastāv no 99 lappusēm, kurās ietilpst 45 attēli, 22 tabulas un 3 pielikumi. 3. pielikums satur programmas pirmkoda fragmentus.

ANNOTATION

The food recipe search and generation web application "FOODYML" has been developed to allow users to conveniently search, filter, and generate food recipes in the Latvian language, taking into account individual dietary restrictions and allergies. The following technologies were used during the development of the system: HTML5, CSS, JavaScript, PHP 8, Laravel, Vue 3, Inertia.js, Bootstrap 5, and MySQL. The DeepSeek artificial intelligence API was used for recipe generation, and the DomPDF library was used for PDF document creation. Laravel Sanctum was used for user authentication, and Vite was used for asset bundling.

The qualification work includes an introduction, task definition, requirements specification, justification for the choice of development tools, a description of software product modelling and design, a description of data structures, and conclusions. The introduction describes the relevance of a recipe search system in the Latvian language. The task definition specifies the tasks the system was required to perform. The requirements specification consists of input and output information, as well as functional and non-functional system requirements. The justification for tool selection identifies the tools used for development and their application. The software product modelling and design description includes the system structure model, data flow model, and functional system model. The data structure description presents the database relation schema, as well as table structures with data types and data lengths. The user guide specifies the system's hardware and software requirements, installation and launch procedures, and application usage instructions. The test section provides a detailed usability testing description with visual illustrations.

The qualification work consists of 99 pages, containing 45 figures, 22 tables, and 3 appendices. The appendices contain fragments of the program's source code.

SATURS

IEVADS.....	5
1. UZDEVUMA NOSTĀDNE.....	6
2. PRASĪBU SPECIFIKĀCIJA.....	7
2.1. Ieejas un izejas informācijas apraksts.....	7
2.1.1. Ieejas informācijas apraksts.....	7
2.1.2. Izejas informācijas apraksts.....	9
2.1.3. Ārējas informācijas apraksts.....	12
2.2. Funkcionālās prasības.....	13
2.3. Nefunkcionālās prasības.....	18
3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS.....	20
4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA.....	22
4.1. Sistēmas struktūras modelis.....	22
4.1.1. Sistēmas arhitektūra.....	22
4.1.2. Sistēmas ER modelis.....	23
4.2. Funkcionālais sistēmas modelis.....	25
4.2.1. Datu plūsmu modelis.....	25
5. DATU STRUKTŪRU APRAKSTS.....	31
6. LIETOTĀJA CEĻVEDIS.....	41
6.1. Sistēmas prasības aparatūrai un programmatūrai.....	41
6.2. Sistēmas instalācija un palaišana.....	41
6.3. Programmas apraksts.....	44
7. PROGRAMMATŪRAS LIETOJAMĪBAS TESTĒŠANA.....	56
NOBEIGUMS.....	58
INFORMĀCIJAS AVOTI.....	59
PIELIKUMI.....	61
1.pielikums.....	61
2.pielikums.....	62
3.pielikums.....	63

IEVADS

Šobrīd ir ļoti svarīgi izmantot vienotus digitalizētus risinājumus, kas ļauj vienkāršot ikdienas procesus, tostarp ēdiena gatavošanas plānošanu un recepšu meklēšanu. Mūsdienās cilvēki arvien biežāk saskaras ar situāciju, kad ledusskapī ir noteikts produktu klāsts, bet nav ideju, ko no tiem pagatavot. Papildus tam daudziem cilvēkiem ir dažādi uztura ierobežojumi, alerģijas, diētas vai veselības traucējumi, kas padara piemērotu recepšu atrašanu vēl sarežģītāku. Lai novērstu šīs problēmas un izvairītos no laikietilpīgas recepšu meklēšanas dažādos interneta resursos, ir nepieciešams izstrādāt tīmekļa lietojumprogrammu, kas apvienotu recepšu meklēšanu, filtrēšanu un personalizētu recepšu ģenerēšanu vienuviet.

Ēdiena recepšu jomā tiešsaistē ir pieejami daudzi resursi angļu valodā, piemēram, Allrecipes, Tasty vai BBC Good Food, taču latviešu valodā šādu rīku klāsts ir ievērojami ierobežotāks. Lielākā daļa esošo latviešu recepšu vietņu piedāvā tikai vienkāršu recepšu katalogu bez iespējas filtrēt pēc uztura ierobežojumiem vai alerģijām, nemaz nerunājot par mākslīgā intelekta izmantošanu recepšu ģenerēšanā. Turklāt šādas platformas bieži nenodrošina iespēju saglabāt iecienītās receptes, vērtēt tās vai lejupielādēt ērtā formātā. Šāda situācija norāda uz būtisku trūkumu digitālo risinājumu tirgū latviešu valodā runājošajiem lietotājiem, kuri gatavo ēdienu mājas apstākļos.

Digitālais risinājums FOODYML ļauj lietotājiem ne tikai meklēt un filtrēt receptes pēc dažādiem kritērijiem, piemēram, ēdienreizes, diētas tipa vai proteīna avota, bet arī ģenerēt pilnīgi jaunas receptes, izmantojot DeepSeek mākslīgā intelekta modeli, balstoties uz lietotāja norādītajām sastāvdaļām. Sistēma ņem vērā katra lietotāja individuālos uztura ierobežojumus un alerģijas, automātiski izslēdzot nepiemērotas receptes no rezultātiem. Starp stiprajām pusēm ir iespēja saglabāt receptes favorītos, novērtēt tās piecu zvaigžņu skalā, kā arī lejupielādēt jebkuru recepti PDF formātā. Šo funkcionalitāti ir iespējams izmantot gan reģistrētiem lietotājiem ar pilnu piekļuvi, gan viesiem ar ierobežotām iespējām, gan administratoriem, kas pārvalda sistēmas saturu. Turklāt sistēma pilnībā darbojas latviešu valodā, kas padara to pieejamu plašākai auditorijai Latvijā.

1. UZDEVUMA NOSTĀDNE

Kvalifikācijas darba uzdevums ir izstrādāt tiešsaistes ēdienu recepšu meklēšanas un ģenerēšanas sistēmu "FOODYML". Sistēma tiks paredzēta lietotājiem, kuri gatavo ēdienu mājas apstākļos un vēlas ātri atrast vai radīt jaunus ēdienu risinājumus latviešu valodā. Programma nodrošinās iespēju gan ģenerēt personalizētas receptes, izmantojot pieejamās sastāvdaļas un mākslīgā intelekta modeli, gan meklēt un filtrēt esošās receptes pēc dažādiem kritērijiem. Sistēma tiks izstrādāta kā tīmekļa lietojumprogramma ar trīs lietotāju lomām: administrators, reģistrēts lietotājs un viesis.

Ir plānotas vairākas funkcijas:

- administratoram būs iespēja pārvaldīt informāciju par sistēmas lietotājiem un receptēm, apskatīt statistiku par recepšu lietojumu un vērtējumiem, kā arī izmantot visas funkcijas, kas pieejamas reģistrētiem lietotājiem;
- reģistrētam lietotājam būs iespēja:
 - ģenerēt receptes pēc sastāvdaļām, izmantojot mākslīgo intelektu;
 - meklēt un filtrēt receptes pēc pagatavošanas laika, sastāvdaļām, kategorijām un uztura nosacījumiem;
 - saglabāt receptes savā profilā un organizēt tās favorītos;
 - novērtēt receptes skalā no 1 līdz 5 un apskatīt gan savus, gan kopējos vērtējumus;
 - rediģēt sava profila informāciju;
 - ģenerēt PDF priekš jebkuras sistēmā esošas receptes;
- viesim būs iespēja ģenerēt receptes pēc sastāvdaļām, meklēt un filtrēt receptes, apskatīt recepšu vērtējumus un aprakstus, ģenerēt PDF priekš jebkuras sistēmā esošas receptes, kā arī reģistrēties sistēmā, lai iegūtu papildu funkcionalitāti.

Informācija par sistēmas lietotājiem un to mijiedarbību ar sistēmu tiks attēlota Lietojumgadījumu diagrammā (USE CASE diagram), kas ir pieejama 1.pielikumā, atspoguļojot detalizētu informāciju sistēmas lietotāju mijiedarbību ar sistēmu.

2. PRASĪBU SPECIFIKĀCIJA

2.1. Ieejas un izejas informācijas apraksts

2.1.1. Ieejas informācijas apraksts

Sistēmā tiks nodrošināta šādas ieejas informācijas, ko ievadīs lietotājs no tastatūras vai izvēles opcijām, apstrāde.

1. Informācija par lietotājiem sastāvēs no šādiem datiem.

- Vārds - lietotāja vārds - burtu teksts ar izmēru līdz 50 rakstzīmēm (piem., "Jānis").
- E-pasts - lietotāja elektroniskā pasta adrese - burtu teksts ar izmēru līdz 100 rakstzīmēm (piem., janis.berzins@epasts.lv).
- Parole - parole, ar kuru lietotājs autorizējas sistēmā - burtu un ciparu teksts ar izmēru no 8 līdz 255 rakstzīmēm (piem., Parole123!).
- Loma - lietotāja loma sistēmā - burtu teksts ar izmēru līdz 30 rakstzīmēm (piem., "administrators", "lietotājs").
- Diētas tips - lietotāja uztura ierobežojuma veids - burtu teksts ar izmēru līdz 30 rakstzīmēm (piem., "vegāns", "veģetārietis", "bezglutēna").
- Alerģija - lietotāja alerģiskā reakcija uz pārtikas produktiem - burtu teksts ar izmēru līdz 50 rakstzīmēm (piem., "rieksti", "piena produkti", "olas").

2. Informācija par receptēm sastāvēs no šādiem datiem.

- Nosaukums - receptes nosaukums - burtu teksts ar izmēru līdz 200 rakstzīmēm (piem., "Itāļu pasta ar tomātu mērci").
- Apraksts - receptes detalizēts apraksts - burtu teksts ar izmēru līdz 1000 rakstzīmēm (piem., "Tradicionāla itāļu virtuve ar svaigu tomātu mērci un baziliku").
- Pagatavošanas laiks - laiks minūtēs - cipars no 1 līdz 999 (piem., 45).
- Porciju skaits - ēdiena daudzums - cipars no 1 līdz 50 (piem., 4).
- Grūtības līmenis - receptes sarežģītība - burtu teksts ar izmēru līdz 20 rakstzīmēm (piem., "viegls", "vidējs", "grūts").
- Ēdienreizes tips - ēdiena kategorija - burtu teksts ar izmēru līdz 30 rakstzīmēm (piem., "pusdienas", "brokastis", "vakariņas").

- Diētas tips - uztura kategorija - burtu teksts ar izmēru līdz 30 rakstzīmēm (piem., "vegāns", "bezglutēna", "keto").
- Olbaltumvielu avots - galvenais proteīna avots - burtu teksts ar izmēru līdz 50 rakstzīmēm (piem., "vista", "zivis", "tofu").
- Publiskuma statuss - receptes pieejamība - burtu teksts (piem., "publiska", "privāta").

3. Informācija par receptes attēliem sastāvēs no šādiem datiem.

- Attēls - receptes vizuālais attēlojums - Base64 kodēts teksts (piem., data:image/png;base64,iVBORw0KG...).

4. Informācija par sastāvdaļām sastāvēs no šādiem datiem.

- Nosaukums - produkta nosaukums - burtu teksts ar izmēru līdz 100 rakstzīmēm (piem., "tomāti", "olīveļļa", "baziliks").
- Kategorija - produkta grupas piederība - burtu teksts ar izmēru līdz 50 rakstzīmēm (piem., "dārzeņi", "gaļa", "garšvielas").

5. Informācija par receptes sastāvdaļām sastāvēs no šādiem datiem.

- Daudzums - produkta daudzums ar mērvienību - burtu un ciparu teksts ar izmēru līdz 50 rakstzīmēm (piem., "500g", "2 ēd.k.", "1 gab.").

6. Informācija par pagatavošanas soļiem sastāvēs no šādiem datiem.

- Soļa apraksts - darbības apraksts - burtu teksts ar izmēru līdz 500 rakstzīmēm (piem., "Uzvārt ūdeni lielā katlīnā un pievienot sāli").
- Soļa numurs - secības kārtas numurs - cipars no 1 līdz 50 (piem., 1, 2, 3).

7. Informācija par vērtējumiem sastāvēs no šādiem datiem.

- Zvaigžņu skaits - receptes novērtējums - cipars no 1 līdz 5 (piem., 4, 5).

8. Informācija par alerģijām sastāvēs no šādiem datiem.

- Nosaukums - alerģijas nosaukums - burtu teksts ar izmēru līdz 50 rakstzīmēm (piem., "rieksti", "olas", "piena produkti").

9. Informācija par diētas ierobežojumiem sastāvēs no šādiem datiem.

- Nosaukums - diētas ierobežojuma nosaukums - burtu teksts ar izmēru līdz 30 rakstzīmēm (piem., "vegāns", "veģetārietis", "bezglutēna", "bez laktozes").

10. Informācija par recepšu meklēšanu un filtrēšanu sastāvēs no šādiem datiem.

- Meklēšanas atslēgvārds - receptes nosaukums vai tā daļa - burtu teksts ar izmēru līdz 100 rakstzīmēm (piem., "vistas zupa", "pasta").
- Ēdienreizes filtrs - ēdienreizes tips filtrēšanai - burtu teksts ar izmēru līdz 30 rakstzīmēm (piem., "brokastis", "pusdienas", "vakariņas").
- Olbaltumvielu avota filtrs - proteīna avots filtrēšanai - burtu teksts ar izmēru līdz 50 rakstzīmēm (piem., "vista", "liellopa gaļa", "pupas").
- Diētas tipa filtrs - uztura kategorija filtrēšanai - burtu teksts ar izmēru līdz 30 rakstzīmēm (piem., "vegāns", "keto", "paleo").

11. Informācija par receptes ģenerēšanu sastāvēs no šādiem datiem.

Receptes ģenerēšanai lietotājs izvēlas parametrus no sistēmā jau esošajām tabulām:

- Sastāvdaļas - izvēle no sastāvdaļu tabulas - viena vai vairākas sastāvdaļas pēc to ID (piem., izvēlas "tomāti", "siers", "makaroni").
- Grūtības pakāpe - izvēle no iepriekš definētiem līmeņiem - burtu teksts (piem., "viegls", "vidējs", "grūts").
- Alerģijas - izvēle no alerģiju tabulas - viena vai vairākas alerģijas pēc to ID (piem., izvēlas "rieksti", "olas").
- Diētas ierobežojumi - izvēle no diētas ierobežojumu tabulas - viens vai vairāki ierobežojumi pēc to ID (piem., izvēlas "vegāns", "bez laktozes").
- Porciju skaits - izvēle no nolaižama saraksta - cipars no 1 līdz 20 (piem., 1, 2, 3, 4).

Papildus šai informācijai, receptes ģenerēšanas funkcionalitātei tiks izmantota informācija no ārējās DeepSeek API platformas, kas atgriezīs strukturētu JSON formātā ģenerēto recepti. Ģenerētā recepte netiks saglabāta datu bāzē, bet tiks tikai attēlota ekrānā pēc JSON atbildes apstrādes.

2.1.2. Izejas informācijas apraksts

1. Recepšu meklēšanas un filtrēšanas rezultātu saraksts ekrānā

Receptes tiks attēlotas kartīšu režģī ar šādu informāciju: lapas augšdaļā virsraksts "Recepšu Meklētājs", zem tā meklēšanas lauks ar viettura tekstu "Meklēt recepti...". Nākamajā rindā trīs nolaižamās izvēlnes filtrēšanai: "Visas ēdienreizes", "Visi uztura veidi" un "Visi olbaltumvielu avoti", blakus sarkana poga "Notīrīt filtrus". Zem filtriem divas papildu nolaižamās izvēlnes: kārtošanas kritērijs un kārtošanas virziens. Rezultāti tiek attēloti kartīšu veidā ar receptes attēlu, nosaukumu un divām iezīmēm apakšā: ēdienreizes tips un uztura veids. Katrai kartītei ir peles uzbraukšanas efekts, kas uzbraucot parāda vidējo vērtējumu.

2. Receptes detalizēts skats ekrānā

Receptes pilnā informācija tiks parādīta ar šādu struktūru: lapas augšdaļā centrēts receptes nosaukums lieliem burtiem. Zem nosaukuma rindā divas ikonas: pulkstenis ar gatavošanas laiku minūtēs un zvaigzne ar sarežģītības līmeni tekstā. Zem tās atsevišķā sadaļā interaktīvas zvaigznes vērtēšanai ar vidējo vērtējumu formātā "4.5 / 5", kā arī ievadlauks komentāram. Nākamajā sadaļā attēlu galerija ar navigācijas bultiņām. Klikšķinot uz attēla, tas atveras pilnekrāna skatā. Zem galerijas īss apraksts. Kreisajā pusē sadaļa "Sastāvdaļas:" ar nolaižamo izvēlni porciju skaitam un sarakstu ar sastāvdaļām, kur katrai norādīts daudzums un nosaukums. Labajā pusē sadaļa "Kā pagatavot:" ar numurētiem soļiem, kur katrs solis ir atsevišķā rindkopā. Zem satura atsauksmju sadaļa. Reģistrētiem lietotājiem apakšā redzama poga saglabāšanai favorītos un poga PDF lejupielādei.

3. PDF izdruka ar receptes pilnu informāciju

Receptes PDF dokuments tiks ģenerēts ar sekojošu struktūru: dokumenta augšdaļā centrēts teksts "FOODYML" treknrakstā un receptes nosaukums lieliem trekniem burtiem melnā krāsā, atdalīts ar treknu horizontālu līniju. Ja receptei ir vērtējums, zem nosaukuma tiks attēlotas piecas zvaigznes (★ aizpildītas vai ☆ tukšas) kopā ar skaitlisko vērtējumu formātā "4.5 / 5.0". Nākamajā sadaļā baltā blokā divās kolonnās tiks izvietota informācija ar trekniem melniem uzrakstiem: gatavošanas laiks, grūtības pakāpe, ēdienreize un uzturs, un pēc vajadzības - diētas tips un olbaltumvielu avots. Ja receptei ir apraksts, tas tiks parādīts baltā blokā ar melnu kreiso malu. Sastāvdaļu sadaļā katrai sastāvdaļai daudzums tiks izcelts treknā melnā krāsā, kam sekos nosaukums. Katrs ieraksts ir atdalīts ar pelēku kreiso malu. Gatavošanas soļu sadaļā katrs solis tiks numurēts automātiski ar trekniem melniem cipariem kreisajā pusē. Dokumenta beigās pelēkā kājēnē tiks norādīts ģenerēšanas datums un laiks, kā arī piezīme par personīgu lietošanu. Dokuments veidots tā, lai to ir ērti printēt.

4. Lietotāja profila kopsavilkums ekrānā

Pirmā sekcija "Konta Informācija": rediģējams vārda lauks un atspējots e-pasta lauks ar piezīmi, ka e-pasts nav maināms. Otrā sekcija paroles maiņai (pašreizējā parole, jaunā parole, apstiprināšana). Trešā sekcija "Diētas Ierobežojumi": izvēles rūtiņu režģis ar visiem pieejamajiem diētas tipiem. Ceturtā sekcija "Alerģijas": izvēles rūtiņu režģis ar visām alerģijām. Apakšā poga "Saglabāt Izmaiņas" un konta dzēšanas poga.

5. Paziņojumi par darbību statusiem ekrānā

Labajā augšējā ekrāna stūrī parādās paziņojumu logs: veiksmīgas darbības gadījumā zaļš fons ar baltu tekstu un atzīmes ikonu, neveiksmīgas darbības gadījumā sarkans fons ar baltu tekstu un brīdinājuma ikonu. Paziņojums automātiski pazūd pēc 5 sekundēm. Piemēri: "Recepte pievienota favorītiem", "Nepareiza parole".

6. Ģenerētās receptes attēlojums ekrānā

Pēc receptes ģenerēšanas rezultāts tiks parādīts tajā pašā lapā sadaļā zem ģenerēšanas paneļa. Tiks parādīts receptes nosaukums, sadaļa "Sastāvdaļas:" ar sarakstu un sadaļa "Pagatavošana:" ar numurētiem soļiem. Apakšā lodziņš ar brīdinājuma ikonu un tekstu "Šī recepte ir ģenerēta, izmantojot mākslīgo intelektu. Lūdzu, pārbaudiet recepti pirms tās pagatavošanas." Poga "Ģenerēt Recepti" ir redzama jau ģenerēšanas panelī ar izvēlēto sastāvdaļu sarakstu.

7. Administratora panelis receptes pārvaldībai ekrānā

Administrators redzēs pārvaldības paneli: augšdaļā virsraksts "Recepšu pārvaldība" un poga "+ Pievienot recepti". Receptes attēlotas kartīšu režģī, kur katrai kartītei redzams attēls, nosaukums un iezīmes (ēdienreize, grūtības pakāpe, gatavošanas laiks minūtēs). Katrai kartītei divas ikonas pogas: rediģēšana (zelta krāsā) un dzēšana (sarkanā krāsā). Dzēšanas apstiprināšana notiek tieši kartītē - parādās pogas "Dzēst!" un "Atcelt".

8. Administratora receptes pievienošanas/rediģēšanas forma ekrānā

Rediģēšanas un pievienošanas forma atveras kā sānu atvilkne (drawer) no labās puses. Galvenes josla satur nosaukumu "Pievienot recepti" vai "Rediģēt recepti" un aizvēršanas pogu. Forma satur: nosaukuma lauku, apraksta lauku, laiku (min) un grūtības pakāpi (2 kolonnas), ēdienreizi un uzturvielu tipu (2 kolonnas), diētas tipu un olbaltumvielu avotu (2 kolonnas), publiskuma izvēles rūtiņu. Tad trīs sadaļas ar atdalītājiem: "Attēli" (attēlu priekšskatījums un augšupielādes poga), "Sastāvdaļas" (dinamiska forma ar + pogu katrai sastāvdaļai: izvēlne,

daudzums, mērvienība) un "Soļi" (numurēti teksta lauki ar + pogu). Apakšā: poga "Atcelt" un poga "Saglabāt". Validācijas kļūdas sarkanas pie laukiem.

9. Iepirkumu saraksta attēlošana ekrānā

Lapa sadalīta divās kolonnās: kreisajā pusē recepšu izvēlētājs ar meklēšanas lauku un ritināmu sarakstu (katrai receptei attēls un nosaukums; klikšķinot pievieno vai noņem no izvēles). Labajā pusē izvēlēto recepšu saraksts ar porciju skaitītāju (– / +) un noņemšanas pogu katrai receptei, un poga "Ģenerēt sarakstu". Pēc ģenerēšanas zemāk parādās iepirkumu saraksts: sastāvdaļas sadalītas pēc kategorijām. Katrai sastāvdaļai ir izvēles rūtiņa, daudzums ar mērvienību treknrakstā un nosaukums. Atzīmējot rūtiņu, gan daudzumam, gan nosaukumam tiek uzlikts pārsvītrojums un tie kļūst gaišāki.

10. Iepirkumu saraksta PDF izdruka

Iepirkumu saraksta PDF dokuments tiks ģenerēts ar sekojošu struktūru: dokumenta augšdaļā centrēts teksts "FOODYML" treknrakstā ar izkārtotiem burtiem, virsraksts "Iepirkumu saraksts" lieliem trekniem burtiem un ģenerēšanas datums ar laiku, atdalīts ar melnu horizontālu svītru. Zem galvenes blokā ar melnu kreiso malu uzskaitītas iekļautās receptes ar to porciju skaitu treknrakstā (piemēram, "Borščs (2x)"). Sastāvdaļas tiek grupētas pa kategorijām - katras kategorijas nosaukums tiek parādīts ar lielajiem burtiem un treknrakstā ar apakšsvītru. Katrā kategorijā katrai sastāvdaļai ir rinda ar atzīmes lodziņu "[]", daudzumu un mērvienību treknrakstā kreisajā pusē un nosaukumu labajā pusē. Dokumenta beigās pelēkā kājenē atkārtots "FOODYML" un ģenerēšanas datums. Dokuments ir pilnībā melnbaltā krāsu shēmā, piemērots drukāšanai.

2.1.3. Ārējas informācijas apraksts

Sistēmā tiks izmantota ārēja informācija no diviem avotiem: DeepSeek API platformas un Google autentifikācijas pakalpojuma.

1. DeepSeek API platforma (recepšu ģenerēšana ar mākslīgo intelektu).

DeepSeek API platforma tiks izmantota, lai ģenerētu jaunas receptes, pamatojoties uz lietotāja ievadītajiem datiem. Sistēma nosūta uz API strukturētu pieprasījumu latviešu valodā, kas satur: lietotāja ievadīto sastāvdaļu sarakstu un (ja lietotājs to izvēlējies) lietotāja uztura ierobežojumu un alerģiju nosaukumus. Pieprasījumā ir iekļauta instrukcija atgriezt atbildi noteiktā strukturētā formātā.

API atgriež strukturētu atbildi ar šādu informāciju:

- Receptes nosaukums - ģenerētās receptes nosaukums - teksts ar izmēru līdz 200 rakstzīmēm (piem., "Vegānā ziedkāpostu krēmzupa ar ķiplokiem").
- Sastāvdaļu saraksts - katra sastāvdaļa satur nosaukumu (teksts ar izmēru līdz 100 rakstzīmēm, piem., "ziedkāposti") un daudzumu ar mērvienību (teksts ar izmēru līdz 50 rakstzīmēm, piem., "500g").
- Pagatavošanas soļi - katrs solis satur kārtas numuru (cipars) un darbības aprakstu (teksts ar izmēru līdz 500 rakstzīmēm, piem., "Sakapā ziedkāpostus sīkos ziediņos un pārskalojot zem tekošā ūdens").

Ja API neatbild vai atgriež kļūdu, sistēma veic atkārtotu mēģinājumu (līdz 2 reizēm). Ģenerētā recepte netiek saglabāta sistēmā un tiek izmantota tikai vienreizējai attēlošanai lietotājam.

2. Google autentifikācijas pakalpojums (OAuth 2.0).

Sistēma izmanto Google OAuth 2.0 protokolu, lai nodrošinātu pieteikšanos ar Google kontu. Savienošanās notiek šādi: lietotājs nospiež pogu "Pieteikties ar Google", sistēma pāradresē lietotāju uz Google autorizācijas lapu, lietotājs apstiprina piekļuvi, un Google atgriež sistēmai šādus datus:

- Lietotāja vārds - teksts (piem., "Jānis Bērziņš").
- E-pasta adrese - teksts (piem., janis@gmail.com).
- Unikāls lietotāja identifikators - teksts, ko piešķir Google, lai viennozīmīgi identificētu lietotāju.

Ja lietotājs ar šādu e-pasta adresi sistēmā vēl nepastāv, tiek izveidots jauns konts. Ja konts jau pastāv, lietotājs tiek pieteikts esošajā kontā.

2.2. Funkcionālās prasības

Sistēmai jānodrošina šādas funkcionālās iespējas:

1. Receptu meklēšana un filtrēšana (skat. 2.2.1. tabulu).

2.2.1.tabula

Identifikators	RM 2.2.1.
Ievads	

Funkcija paredzēta, lai lietotājs varētu meklēt un filtrēt receptes pēc dažādiem kritērijiem, kā arī apskatīt detalizētu receptes informāciju.

Ievaddati

1. Meklēšanas atslēgvārds - neobligāts lauks, kurā lietotājs ievada nosaukumu vai sastāvdaļu.
2. Kategorija - neobligāts lauks, kurā lietotājs izvēlas ēdiena kategoriju.
3. Grūtības pakāpe - neobligāts lauks ar izvēles vērtībām (viegls, vidējs, sarežģīts).
4. Olbaltumvielu avots - neobligāts lauks (gaļa, zivis, pākšaugi u.c.).
5. Kārtošanas lauks - neobligāts lauks (gatavošanas laiks, vērtējums, grūtības pakāpe)
6. Kārtošanas virziens - neobligāts lauks (dilstoši, augoši)

Apstrāde

1. Sistēma saņem lietotāja ievadītos meklēšanas un filtrēšanas parametrus.
2. Sistēma veic vaicājumu datubāzē, atlasot receptes, kas atbilst norādītajiem kritērijiem.
3. Sistēma sakārto rezultātus pēc norādītajiem kārtošanas kritērijiem, ja to nav tad pēc datu bāzē glabātā identifikatora.

Izvaddati

1. Ja atrasti rezultāti, sistēma atgriež recepšu sarakstu ar pamatinformāciju.
2. Izvēloties recepti, sistēma attēlo detalizētu informāciju: aprakstu, sastāvdaļas, pagatavošanas soļus un laika patēriņu.
3. Ja nav atrasti rezultāti, tiek parādīts paziņojums par tukšu rezultātu.

Kļūdas paziņojumi

1. Neviena recepte neatbilst meklēšanas kritērijiem.

Izsaucamās prasības

Funkciju var izsaukt jebkurš sistēmas lietotājs (gan reģistrēts, gan neregistrēts).

2. Recepšu ģenerēšana (skat. 2.2.2. tabulu).

2.2.2.tabula

<i>Identifikators</i>	RG 2.2.2.
<i>Ievads</i>	
Funkcija paredzēta, lai sistēma ģenerētu receptes latviešu valodā, balstoties uz lietotāja norādītajām sastāvdaļām un ierobežojumiem.	

<i>Ievaddati</i>
1. Pieejamās sastāvdaļas - obligāts lauks, kurā lietotājs norāda vismaz 3 sastāvdaļas. 2. Diētas ierobežojumi - neobligāts lauks (vegāns, bezglutēna u.c.). 3. Vēlāmais ēdiena veids - neobligāts lauks (brokastis, pusdienas, vakariņas, uzkoda).
<i>Apstrāde</i>
1. Sistēma saņem lietotāja ievadītās sastāvdaļas un ierobežojumus. 2. Sistēma nosūta pieprasījumu mākslīgā intelekta API receptes ģenerēšanai. 3. Sistēma apstrādā atgriezto recepti, kas ir JSON formātā un formatē to.
<i>Izvaddati</i>
1. Ja ģenerēšana veiksmīga, sistēma atgriež recepti ar pilnu struktūru: nosaukumu, aprakstu, sastāvdaļām un pagatavošanas soļiem. 2. Ja ģenerēšana neveiksmīga, tiek parādīts kļūdas paziņojums.
<i>Kļūdas paziņojumi</i>
1. Mākslīgā intelekta API kļūda - recepti nevar ģenerēt.
<i>Izsaucamās prasības</i>
Funkciju var izsaukt jebkurš sistēmas lietotājs, kuram ir piekļuve internetam.

3. Receptes pievienošana (administrators) (skat. 2.2.3. tabulu).

2.2.3.tabula

<i>Identifikators</i>	RP 2.2.3
<i>Ievads</i>	
Funkcija paredzēta, lai administrators varētu pievienot jaunas receptes sistēmas datubāzei.	
<i>Ievaddati</i>	
1. Nosaukums - obligāts lauks, teksta formātā. 2. Apraksts - obligāts lauks, teksta formātā. 3. Sastāvdaļu saraksts - obligāts lauks, saraksts ar nosaukumiem un daudzumiem. 4. Pagatavošanas soļi - obligāts lauks, numurēts soļu saraksts. 5. Pagatavošanas laiks - obligāts lauks, vesels skaitlis minūtēs. 6. Grūtības pakāpe - obligāts lauks, izvēle no saraksta (viegls, vidējs, grūts).	

7. Kategorijas - obligāts lauks, viena vērtība no saraksta. 8. Uztura tips - obligāts lauks, viena vērtība no saraksta (vegāns, bezglutēna u.c.). 9. Attēls - obligāts lauks rediģēšanai, attēla fails (JPG, PNG).
Apstrāde
1. Sistēma pārbauda administratora tiesības. 2. Sistēma validē visus ievadītos datus. 3. Sistēma saglabā recepti datubāzē.
Izvaddati
1. Ja pievienošana veiksmīga, tiek parādīts apstiprinājums un jaunā recepte. 2. Ja pievienošana neveiksmīga, tiek parādīts kļūdas paziņojums.
Kļūdas paziņojumi
1. Nav aizpildīti visi obligātie lauki. 2. Recepte ar šādu nosaukumu jau eksistē. 3. Neatbalstīts attēla formāts. 4. Nav tiesību veikt šo darbību.
Izsaucamās prasības
Funkciju var izsaukt tikai lietotājs ar administratora lomu.

4. Receptes rediģēšana un dzēšana (administrators) (skat. 2.2.4. tabulu).

2.2.4.tabula

Identifikators	RR 2.2.4
Ievads	
Funkcija paredzēta, lai administrators varētu modificēt esošas receptes informāciju vai pilnībā dzēst recepti no sistēmas datubāzes.	
Ievaddati	
1. Nosaukums - obligāts lauks, teksta formātā. 2. Apraksts - obligāts lauks, teksta formātā. 3. Sastāvdaļu saraksts - obligāts lauks, saraksts ar nosaukumiem un daudzumiem. 4. Pagatavošanas soļi - obligāts lauks, numurēts soļu saraksts. 5. Pagatavošanas laiks - obligāts lauks, vesels skaitlis minūtēs. 6. Grūtības pakāpe - obligāts lauks, izvēle no saraksta (viegls, vidējs, grūts). 7. Kategorijas - obligāts lauks, viena vērtība no saraksta. 8. Uztura tips - obligāts lauks, viena vērtība no saraksta (vegāns, bezglutēna u.c.).	

9. Attēls - obligāts lauks rediģēšanai, attēla fails (JPG, PNG). 10. Darbības veids - obligāts lauks, izvēle: "Saglabāt izmaiņas" vai "Dzēst recepti".
<i>Apstrāde</i>
1. Sistēma pārbauda, vai lietotājam ir administratora tiesības. 2. Ja izvēlēta rediģēšana: sistēma validē ievadītos datus un atjaunina tikai tos laukus, kas tika mainīti. 3. Ja izvēlēta dzēšana: sistēma pieprasa apstiprinājumu un dzēš recepti kopā ar saistītajiem vērtējumiem.
<i>Izvaddati</i>
1. Ja rediģēšana veiksmīga, tiek parādīts paziņojums "Recepte veiksmīgi atjaunināta" un attēlota atjauninātā recepte. 2. Ja dzēšana veiksmīga, tiek parādīts paziņojums "Recepte veiksmīgi dzēsta" un administrators tiek novirzīts uz recepšu sarakstu. 3. Ja darbība neveiksmīga, tiek parādīts atbilstošs kļūdas paziņojums.
<i>Kļūdas paziņojumi</i>
1. Nav tiesību veikt šo darbību. 2. Nederīgs datu formāts. 3. Neatbalstīts attēla formāts. 4. Nav aizpildīti visi obligātie lauki.
<i>Izsaucamās prasības</i>
Funkciju var izsaukt tikai lietotājs ar administratora lomu.

4. Lietotāju pārvaldība

- sistēma atbalsta lietotāju reģistrāciju un autentifikāciju;
- reģistrēti lietotāji var rediģēt profilinformāciju;
- administrators var mainīt vai dzēst lietotāju datus.

5. Recepšu pārvaldība

- reģistrēti lietotāji var saglabāt receptes savā kontā;
- reģistrēti lietotāji var pievienot vērtējumus receptēm.

6. Vērtēšanas un atsauksmju sistēma

- lietotāji var piešķirt receptēm vērtējumu skalā 1-5;
- sistēma aprēķina vidējo vērtējumu;

- vidējie vērtējumi ir redzami visiem lietotājiem.

2.3. Nefunkcionālās prasības

1. Sistēmas saskarnes valodai jābūt latviešu valodai ar pareizu latviešu valodas burtu attēlošanu (ā, č, ē, ģ, ī, ķ, ļ, ņ, š, ū, ž).
2. Jānodrošina tīmekļa lietojumprogrammas dinamisks dizains, kas automātiski pielāgojas dažādiem ekrāna izmēriem:
 1. Mobilie telefoni: 320px - 480px
 2. Planšetes: 481px - 768px
 3. Klēpjatori: 769px - 1024px
 4. Datori: 1025px un vairāk
3. Sistēmai jābūt saderīgai ar populārākajiem tīmekļa pārlūkiem: Google Chrome, Mozilla Firefox un Safari.
4. Sistēmai jāsniedz lietotājam informatīvus paziņojumus par veiksmīgiem vai neveiksmīgiem procesiem:
 1. Zaļi paziņojumi veiksmīgām darbībām
 2. Sarkani paziņojumi neveiksmīgām darbībām vai kļūdām
 3. Paziņojumiem jāparādās labajā augšējā stūrī
 4. Paziņojumiem jāpazūd automātiski pēc 5 sekundēm
5. Parolēm jābūt šifrētām datu bāzē.
6. Attēliem jābūt saglabātiem Base64 formātā atsevišķā datu bāzes tabulā.
7. Receptes kartītēm jābūt ar noapaļotiem stūriem un ēnu efektu, lai nodrošinātu modernu izskatu.
8. Navigācijas izvēlei jābūt fiksētai lapas augšdaļā ar sistēmas logo kreisajā pusē.
9. Sistēmas logotipam jābūt redzamam visās lapās kājenē (izņemot reģistrācijas un autentifikācijas skatā).
10. Lietotāja saskarnei jābūt intuitīvai un viegli izmantojamai arī lietotājiem bez tehniskām zināšanām.
11. Lietotāja pieslēgšanās formai jāizskatās sekojoši (skat. 1.att)



A sketch of a user login form. It features a title 'Pieslēgšanās' at the top. Below the title are two input fields: the first is labeled 'E-pasts' and the second is labeled 'Parole'. At the bottom of the form is a large black button with the white text 'Ienākt'.

1. att. Lietotāja pieslēgšanās formas skice

12. Apstiprinājuma modālajam logam jāizskatās sekojoši (skat. 2.att.)



A sketch of a confirmation modal dialog. It has a title 'Apstiprināt'. Below the title is a question in Latvian: 'Sekojošā darbība ir neatgriezeniska, vai tiešām vēlaties turpināt?'. At the bottom, there are two buttons: 'Labi' and 'Atcelt'.

2. att. Apstiprinājuma modālā loga skice

3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS

Recepšu tīmekļa lietojumprogramma ir paredzēta recepšu meklēšanai, filtrēšanai, vērtēšanai un ģenerēšanai ar mākslīgā intelekta palīdzību. Tā sastāv no divām galvenajām daļām: lietotāja daļas (frontend) un servera daļas (backend). Sistēma ir paredzēta izmantošanai pārlūkprogrammās, nodrošinot responsīvu dizainu gan galddatoru, gan mobilo ierīču ekrāniem. Projekts ietver mākslīgā intelekta elementus - recepšu ģenerēšanu, balstoties uz lietotāja norādītajām sastāvdaļām, kā arī datu organizēšanu relāciju datubāzē un PDF dokumentu ģenerēšanu.

Kā galvenā izstrādes vide tiek izmantots **Visual Studio Code (VS Code)**, kas ir bezmaksas, viegls un paplašināms koda redaktors ar daudz paplašināšanas iespējām, iebūvētu termināli un Git integrāciju. VS Code nodrošina ērtu darbu gan ar PHP, gan ar JavaScript un Vue failiem vienuviet. Datubāzes pārvaldībai un vaicājumu izpildei tiek izmantots **DBeaver** - universāls datubāzes administrēšanas rīks, kas atbalsta MySQL un piedāvā vizuālu tabulu struktūras pārskatīšanu, datu rediģēšanu un SQL vaicājumu izpildi.

Servera puses izstrādei tiek izmantota programmēšanas valoda **PHP 8.2**, kas nodrošina modernu sintaksi un augstu veiktspēju. Kā galvenais PHP ietvars tiek izmantots **Laravel 12.0**, kas piedāvā ērtu maršrutēšanu, Eloquent ORM datubāzes vaicājumiem, middleware, validāciju, sesiju pārvaldību un e-pasta funkcionalitāti. E-pasta izsūtīšana tiek apstrādāta asinhronā rindā, izmantojot Laravel Queue sistēmu, kas novērš servera aizkavēšanu liela saņēmēju skaita gadījumā. Datu glabāšanai tiek izmantota relāciju datubāze **MySQL**, kurai piekļūst netieši caur Laravel Eloquent ORM un migrāciju sistēmu.

Lietotāja puses izstrādei tiek izmantots **Vue 3** (versija 3.4.0) - progresīvs JavaScript ietvars, kas nodrošina reaktīvu lietotāja saskarni ar komponentu arhitektūru. Savienojumam starp Laravel servera pusi un Vue frontend tiek izmantots **Inertia.js 2.0**, kas ļauj veidot vienas lapas lietojumprogrammu (SPA) bez nepieciešamības veidot atsevišķu REST API. Vizuālajam noformējumam tiek izmantots **Bootstrap 5.3.8**, kas piedāvā responsīvu grid sistēmu un utilitātklases. Frontend resursu kompilēšanai un izstrādes servera nodrošināšanai tiek izmantots **Vite 6.0.11**.

Skatu slānī tiek izmantots **HTML** kopā ar Laravel **Blade** veidņu dzinēju galvenajam izkārtojumam un PDF veidnēm, savukārt stils tiek nodrošināts ar **CSS** (scoped stils katrā Vue komponentē). HTTP pieprasījumiem no frontend uz serveri tiek izmantota bibliotēka **axios 1.7.4**.

Autentifikācijas un drošības nodrošināšanai tiek izmantots **Laravel Sanctum 4.0**, kas aizsargā pieprasījumus ar CSRF tokeniem un sesiju autentifikāciju. Pieteikšanās ar Google kontu

tiek realizēta ar **Laravel Socialite 5.25** bibliotēku, kas nodrošina OAuth 2.0 integrāciju. Sākotnējā autentifikācijas struktūra (pieteikšanās, reģistrācija, profils) tika ģenerēta ar **Laravel Breeze 2.3** un pēc tam pielāgota projekta vajadzībām.

PDF dokumentu ģenerēšanai tiek izmantota bibliotēka **barryvdh/laravel-dompdf 3.1**, kas ļauj izveidot PDF failus no Blade veidnēm un piedāvāt tos lejupielādei.

Mākslīgā intelekta recepšu ģenerēšanai tiek izmantots **DeepSeek API** - ārējs pakalpojums, kuram tiek nosūtīts lietotāja norādīto sastāvdaļu saraksts un ierobežojumi, un atpakaļ tiek saņemta ģenerēta recepte strukturētā formātā.

Papildu palīgbibliotēkas: **tightenco/ziggy 2.0** eksportē Laravel maršrutu nosaukumus uz JavaScript vidi, **laravel-lang/publisher 16.6** nodrošina validācijas kļūdu paziņojumu tulkošanu latviešu valodā, **@popperjs/core 2.11.8** nodrošina Bootstrap dropdown elementu pozicionēšanu.

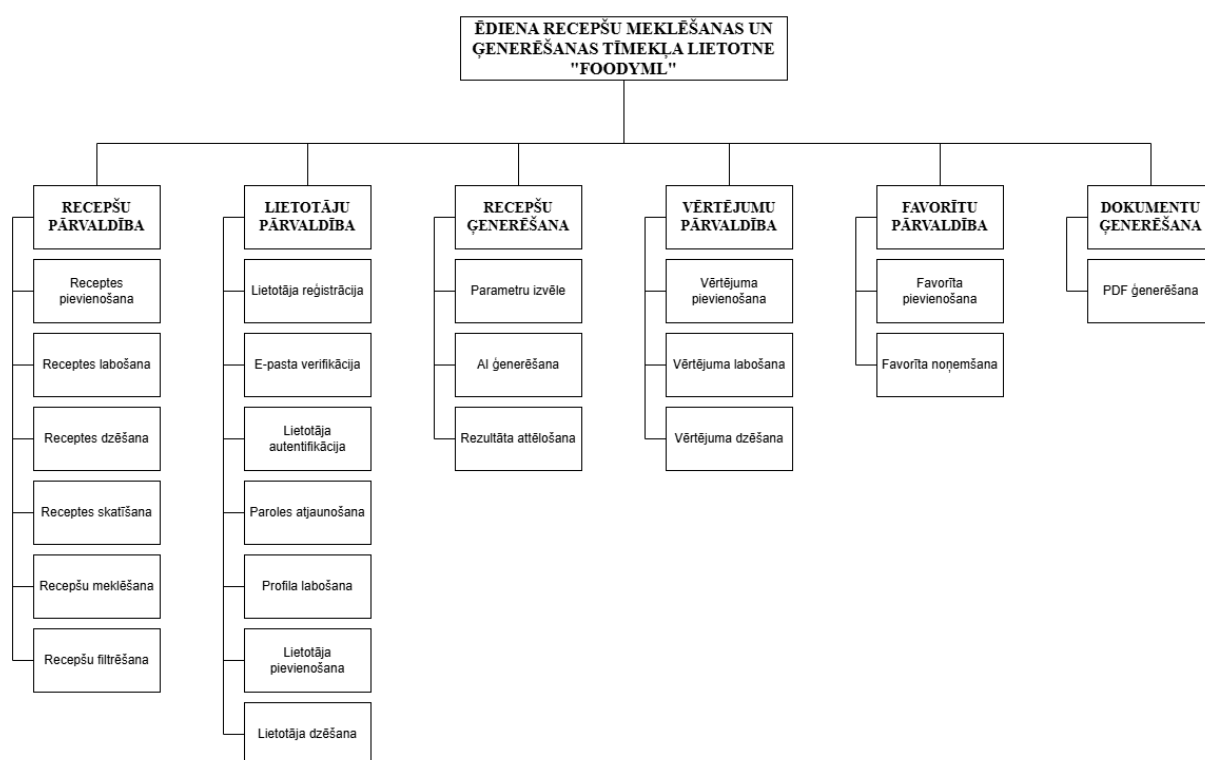
Izstrādes rīki: koda formatēšanai tiek izmantots **Laravel Pint 1.13** (PSR-12 standarts), testēšanai - **PHPUnit 11.5.3**, testdatu ģenerēšanai - **FakerPHP 1.23**, un reāllaika žurnālu skatīšanai - **Laravel Pail 1.2.2**. Paralēlai izstrādes serveru palaišanai tiek izmantots **concurrently 9.0.1**.

4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA

4.1. Sistēmas struktūras modelis

4.1.1. Sistēmas arhitektūra

Sistēma tiks veidota no sešām apakšsistēmām (skat. 3.att.): receptu pārvaldības apakšsistēmas, lietotāju pārvaldības apakšsistēmas, receptu ģenerēšanas apakšsistēmas, vērtējumu pārvaldības apakšsistēmas, favorītu pārvaldības apakšsistēmas un dokumentu ģenerēšanas apakšsistēmas.



3.att. Funkcionālās dekompozīcijas diagramma.

Recepšu pārvaldība - apakšsistēma, kas nodrošina visas darbības ar receptēm sistēmā. Tā ietver šādus moduļus:

- Receptes pievienošana - ļauj administratoram pievienot jaunu recepti ar visiem datiem;
- Receptes labošana - ļauj administratoram rediģēt esošas receptes informāciju;
- Receptes dzēšana - ļauj administratoram dzēst recepti no sistēmas;
- Receptes skatīšana - ļauj visiem lietotājiem apskatīt pilnu receptes informāciju, tostarp attēlus, sastāvdaļas un gatavošanas soļus;
- Recepšu meklēšana - ļauj visiem lietotājiem meklēt receptes pēc atslēgvārdiem;
- Recepšu filtrēšana - ļauj visiem lietotājiem filtrēt receptes pēc dažādiem kritērijiem.

Lietotāju pārvaldība - apakšsistēma, kas nodrošina lietotāju kontu pārvaldību. Tā ietver šādus moduļus:

- Lietotāja reģistrācija - ļauj viesim izveidot jaunu kontu sistēmā;
- E-pasta verifikācija - pēc reģistrācijas lietotājam jāapstiprina savs e-pasts, lai piekļūtu sistēmas funkcijām;
- Lietotāja autentifikācija - ļauj lietotājam pieteikties sistēmā;
- Paroles atjaunošana - ļauj lietotājam atjaunot aizmirstu paroli, izmantojot e-pastu;
- Profila labošana - ļauj lietotājam mainīt sava profila informāciju un uztura ierobežojumus;
- Lietotāja pievienošana - ļauj administratoram izveidot jaunu lietotāja kontu sistēmā;
- Lietotāja dzēšana - ļauj administratoram vai lietotājam dzēst kontu.

Recepšu ģenerēšana - apakšsistēma, kas nodrošina jaunu recepšu ģenerēšanu ar mākslīgo intelektu. Tā ietver šādus moduļus:

- Parametru izvēle - ļauj lietotājam izvēlēties sastāvdaļas un ierobežojumus;
- AI ģenerēšana - nosūta pieprasījumu DeepSeek API un apstrādā atbildi;
- Rezultāta attēlošana - formatē un parāda ģenerēto recepti lietotājam.

Vērtējumu pārvaldība - apakšsistēma, kas nodrošina recepšu novērtēšanu. Tā ietver šādus moduļus:

- Vērtējuma pievienošana - ļauj reģistrētam lietotājam novērtēt recepti un pievienot komentāru;
- Vērtējuma labošana - ļauj lietotājam mainīt savu vērtējumu;
- Vērtējuma dzēšana - ļauj lietotājam dzēst savu vērtējumu.

Favorītu pārvaldība - apakšsistēma, kas nodrošina recepšu saglabāšanu lietotāja profilā. Tā ietver šādus moduļus:

- Favorīta pievienošana - ļauj lietotājam saglabāt recepti favorītos;
- Favorīta noņemšana - ļauj lietotājam noņemt recepti no favorītiem.

Dokumentu ģenerēšana - apakšsistēma, kas nodrošina PDF dokumentu izveidošanu. Tā ietver šādu moduli:

- PDF ģenerēšana - ļauj lietotājam lejupielādēt recepti PDF formātā.

4.1.2. Sistēmas ER modelis

Datu bāzes projektēšanā datu kopu un saišu starp tām attēlošanai tika lietota realitāšu saišu diagramma, kas sastāv no divu veidu objektiem - entītēm un relācijām. ER diagramma (sk. 2.pielikumā) sastāv no 11 entītijām, kas atspoguļo datu apstrādi sistēmā.

- **Lietotājs** - uzskaita sistēmas reģistrētos lietotājus ar informāciju par vārdu, e-pastu, reģistrācijas datumu un pēdējo pieteikšanos;
- **Recepte** - glabā informāciju par ēdiena receptēm: nosaukums, apraksts, gatavošanas laiks, sarežģītības pakāpe, ēdienreize, uztura tips, diētas tips un olbaltumvielu avots;
- **Sastāvdaļa** - uzskaita visas pieejamās sastāvdaļas ar to nosaukumiem un kategorijām;

- **Gatavošanas solis** - apraksta receptes gatavošanas instrukcijas secīgos soļos ar soļa numuru un aprakstu;
- **Vērtējums** - glabā lietotāju vērtējumus receptēm ar vērtējuma atzīmi un izvēles komentāru;
- **Uztura ierobežojums** - uzskaita dažādus uztura ierobežojumus (piemēram, laktozes nepanesība, sarkanās gaļas izslēgšana);
- **Alerģija** - glabā informāciju par alerģiju veidiem (piemēram, riekstu alerģija, jūras velšu alerģija);
- **Attēls** - uzskaita receptēm pievienotos attēlus base64 formātā ar faila tipu un izmēru;
- **Loma** - glabā lietotāju iespējamās lomas nosaukumus;

Datu bāzes relācijas parāda, kā divas vai vairākas entītijas ir savstarpēji saistītas:

- starp entītijām "**Lietotājs**" un "**Vērtējums**" ir attiecība viens pret daudziem, jo viens lietotājs var dot vērtējumu daudzām receptēm, bet viens konkrēts vērtējums pieder tikai vienam lietotājam;
- starp entītijām "**Recepte**" un "**Vērtējums**" ir attiecība viens pret daudziem, jo viena recepte var saņemt daudzus vērtējumus no dažādiem lietotājiem, bet viens vērtējums attiecas tikai uz vienu konkrētu recepti;
- starp entītijām "**Lietotājs**" un "**Recepte**" ir attiecība daudzi pret daudziem caur favorītiem, jo viens lietotājs var pievienot daudzas receptes favorītos, bet viena recepte var būt favorītos daudziem lietotājiem;
- starp entītijām "**Lietotājs**" un "**Loma**" ir attiecība viens pret daudziem, jo vienam lietotājam var būt tikai viena loma, bet viena loma var būt vairākiem lietotājiem.
- starp entītijām "**Recepte**" un "**Gatavošanas solis**" ir attiecība viens pret daudziem, jo viena recepte sastāv no daudziem gatavošanas soļiem, bet viens gatavošanas solis pieder tikai vienai receptei;
- starp entītijām "**Recepte**" un "**Sastāvdaļa**" ir attiecība daudzi pret daudziem, jo viena recepte satur daudzas sastāvdaļas, bet viena sastāvdaļa var tikt izmantota daudzās receptēs;
- starp entītijām "**Attēls**" un "**Recepte**" ir attiecība viens pret vienu, jo viens attēls var tikt izmantots vienai receptēm un vienai receptei ir tikai viens attēls;
- starp entītijām "**Lietotājs**" un "**Uztura ierobežojums**" ir attiecība daudzi pret daudziem, jo vienam lietotājam var būt vairāki uztura ierobežojumi, bet viens uztura ierobežojums var attiekties uz daudziem lietotājiem;
- starp entītijām "**Lietotājs**" un "**Alerģija**" ir attiecība daudzi pret daudziem, jo vienam lietotājam var būt vairākas alerģijas, bet viena alerģija var būt daudziem lietotājiem;

- starp entītijām **"Uztura ierobežojums"** un **"Sastāvdaļa"** ir attiecība daudzi pret daudziem, jo viens uztura ierobežojums var aizliegt vairākas sastāvdaļas, bet viena sastāvdaļa var būt aizliegta ar vairākiem uztura ierobežojumiem;
- starp entītijām **"Alerģija"** un **"Sastāvdaļa"** ir attiecība daudzi pret daudziem, jo viena alerģija var attiekties uz vairākām sastāvdaļām, bet viena sastāvdaļa var izraisīt vairākas dažādas alerģijas.

4.2. Funkcionālais sistēmas modelis

4.2.1. Datu plūsmu modelis

Šajā sadaļā tiek detalizēti aprakstīti svarīgākie datu transformācijas un apstrādes procesi, kas notiek sistēmas moduļos. Katram modulim ir izveidots tekstuāls apraksts un datu plūsmas diagramma (DFD).

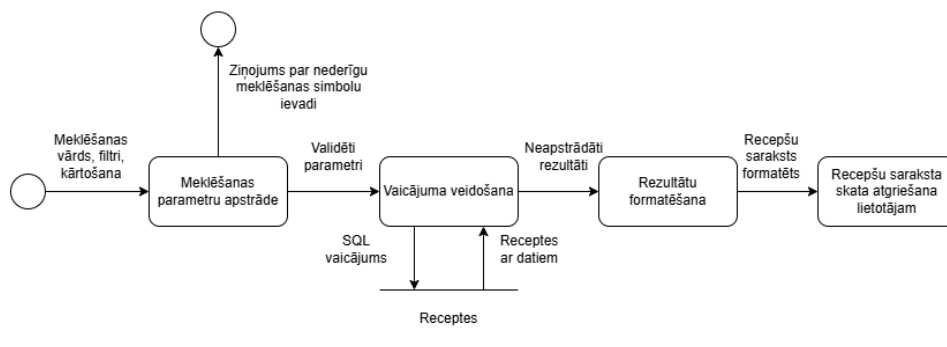
1. Receptu meklēšanas un filtrēšanas modulis (skat. 4.att.).

Lietotājs ievada meklēšanas vārdu, filtru vērtības (ēdienreize, uztura tips, proteīna avots), kārtēšanas parametrus un lapas numuru. Pārlūkprogramma vienlaicīgi nosūta trīs pieprasījumus serverim:

- pieprasījumu pēc receptu saraksta ar norādītajiem filtriem, kārtēšanu un lapas numuru (GET /api/recipes);
- pieprasījumu pēc filtru opcijām - ēdienreizi, uztura tipu un proteīna avotu nosaukumiem (GET /api/recipe-filters);
- pieprasījumu pēc kārtēšanas opcijām un lauku nosaukumiem (GET /api/config).

Serverī tiek veikta parametru validācija un veidots datu bāzes vaicājums. Filtrēšana notiek, pārbaudot, vai receptei ir saistība ar norādīto ēdienreizi, uztura tipu vai proteīna avotu (izmantojot SQL apakšvaicājumus ar EXISTS pret attiecīgajām tabulām). Vienlaikus tiek aprēķināts katras receptes vidējais vērtējums no vērtējumu tabulas (SQL: SELECT AVG(rating) FROM ratings WHERE recipe_id = ...). Tiek ielādēti arī saistītie dati - sastāvdaļas, attēli, sarežģītības līmenis un citi klasifikācijas lauki. Rezultāti tiek sadalīti pa lapām un formatēti.

Lietotājam tiek atgriezts receptu saraksts ar attēliem, vidējiem vērtējumiem un tagiem (ēdienreize, diētas tips), kā arī informācija par pašreizējo lapu, kopējo lapu skaitu un kopējo rezultātu skaitu.

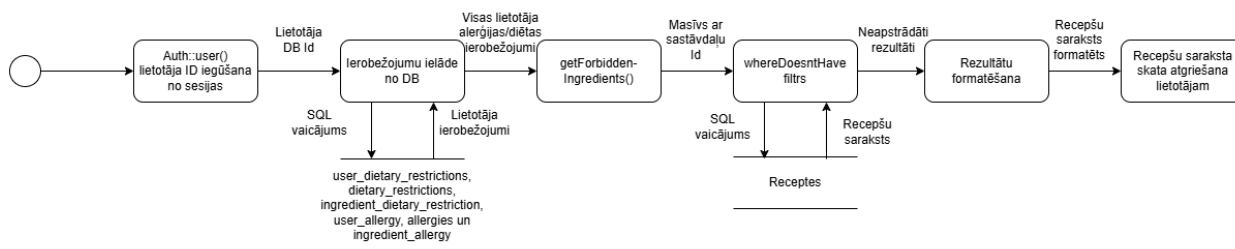


4.att. Receptu meklēšanas un filtrēšanas moduļa datu plūsmas diagramma

2. Personalizētas receptu filtrēšanas modulis (skat. 5.att.).

No sesijas tiek iegūts lietotāja ID. Tad sistēma no datu bāzes ielādē lietotāja uztura ierobežojumus un alerģijas - tiek veikti vaicājumi, kas sasaista lietotāju ar viņa ierobežojumiem (SQL: SELECT ... FROM dietary_restrictions JOIN user_dietary_restrictions WHERE user_id = ...) un ar viņa alerģijām (SQL: SELECT ... FROM allergies JOIN user_allergy WHERE user_id = ...). Katram ierobežojumam un alerģijai tiek ielādēts ar tām saistīto aizliegto sastāvdaļu saraksts no attiecīgajām sasaistes tabulām (ingredient_dietary_restriction un ingredient_allergy).

Metode getForbiddenIngredientIds() apvieno visas aizliegtās sastāvdaļas vienā sarakstā. Pēc tam receptes tiek filtrētas, izslēdzot visas tās, kurās ir kāda no aizliegtajām sastāvdaļām (SQL: SELECT * FROM recipes WHERE NOT EXISTS (SELECT 1 FROM recipe_ingredients WHERE recipe_id = recipes.id AND ingredient_id IN (aizliegto sastāvdaļu saraksts))). Lietotājam tiek atgriezts receptu saraksts, kurā nav nevienas viņam nepiemērotas sastāvdaļas.

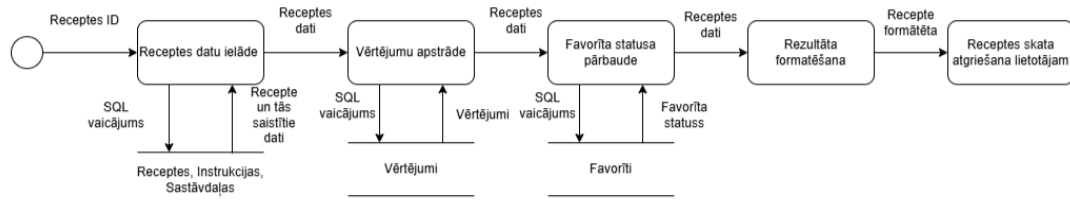


5.att. Personalizētas receptu filtrēšanas moduļa datu plūsmas diagramma

3. Receptes detaļu ielādes modulis (skat. 6.att.).

Lietotājs norāda receptes ID. Pārlūkprogramma nosūta pieprasījumu serverim (GET /api/recipes/{id}). Serverī tiek veikts datu bāzes vaicājums, kas atlasa konkrēto recepti un vienlaikus aprēķina tās vidējo vērtējumu (SQL: SELECT recipes.*, (SELECT AVG(rating) FROM ratings WHERE recipe_id = recipes.id) AS average_rating FROM recipes WHERE id = ...). Kopā tiek ielādēti visi saistītie dati - gatavošanas soļi, sastāvdaļas ar kategorijām, vērtējumi ar lietotāju vārdiem, attēli, sarežģītības līmenis un klasifikācijas lauki.

Sistēma nosaka lietotāja personīgo vērtējumu, pārbaudot, vai vērtējumu sarakstā ir ieraksts ar šī lietotāja ID, un pārbauda, vai recepte ir lietotāja favorītos. Lietotājam tiek atgriezts pilns receptes objekts ar visiem datiem - nosaukumu, aprakstu, attēliem, sastāvdaļām, gatavošanas soļiem, vērtējumiem un komentāriem.



6.att. Receptes detaļu ielādes moduļa datu plūsmas diagramma

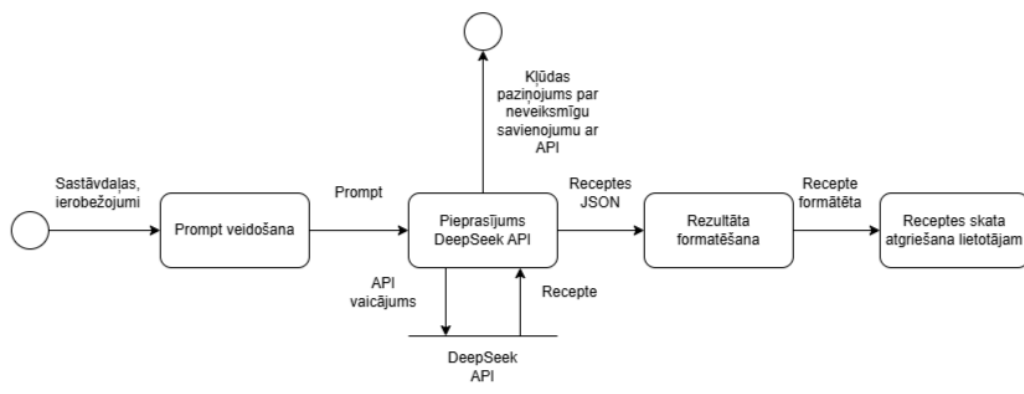
4. AI recepšu ģenerēšanas modulis (skat. 7.att.).

Lietotājs izvēlas sastāvdaļas un norāda, vai izmantot ar viņu saistītos uztura ierobežojumus un alerģijas. Pārlūkprogramma nosūta pieprasījumu serverim (POST /api/generate-recipe) ar izvēlēto sastāvdaļu sarakstu un norādi par preferenču izmantošanu.

Ja lietotājs ir izvēlējis izmantot savas preferences un ir autentificēts, serverī tiek ielādēti viņa ierobežojumi un alerģijas no datu bāzes (SQL: SELECT name FROM dietary_restrictions JOIN user_dietary_restrictions WHERE user_id = ...; SELECT name FROM allergies JOIN user_allergy WHERE user_id = ...).

Tiek veidots strukturēts pieprasījums (prompt) latviešu valodā ar norādītajām sastāvdaļām un obligātu izvades formātu (Nosaukums / Sastāvdaļas / Pagatavošana), kas tiek papildināts ar lietotāja ierobežojumiem un alerģijām, ja tādas ir. Tad tiek nosūtīts pieprasījums uz ārējo mākslīgā intelekta servisu DeepSeek API (POST <https://api.deepseek.com/chat/completions>) ar modeli deepseek-chat, radošuma parametru (temperature: 0.7), maksimālo atbildes garumu (max_tokens: 1500) un sistēmas ziņojumu "Tu esi pavārs. Atbildi tikai latviski."

Ja API neatbild vai atgriež kļūdu, tiek veikts atkārtots mēģinājums līdz 2 reizēm ar 1 sekundes pauzi starp tiem. Saņemtā atbilde tiek iegūta no API atgrieztā datu lauka choices[0].message.content. Lietotājam tiek atgriezta ģenerētās receptes teksts vai kļūdas ziņojums.

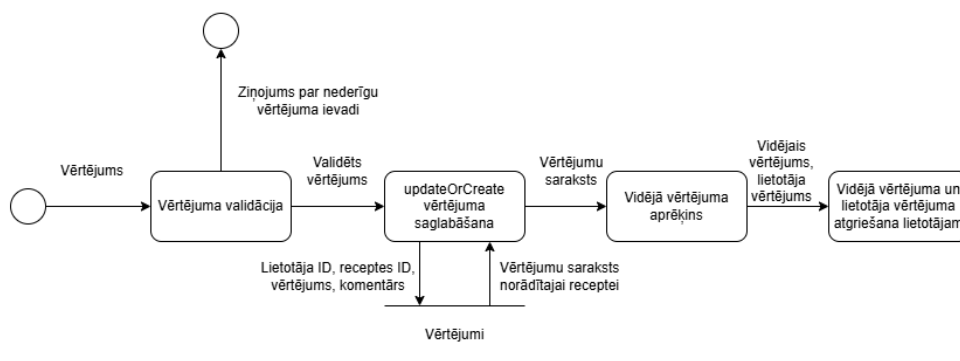


7.att. AI recepšu ģenerēšanas moduļa datu plūsmas diagramma

5. Receptes vērtēšanas modulis (skat. 8.att.).

Lietotājs ievada vērtējumu skalā no 1 līdz 5 konkrētai receptei un pievieno neobligātu komentāru. Pārlūkprogramma nosūta pieprasījumu serverim (POST /ratings) ar receptes ID, vērtējumu un komentāru.

Tiek veikta validācija - pārbaudīts, vai norādītā recepte eksistē datu bāzē, vai vērtējums ir vesels skaitlis no 1 līdz 5, un vai komentārs atbilst prasībām. Pēc tam datu bāzē tiek meklēts, vai šis lietotājs jau ir vērtējis šo recepti (SQL: SELECT * FROM ratings WHERE user_id = ... AND recipe_id = ...). Ja ieraksts jau pastāv, tas tiek atjaunināts ar jaunajām vērtībām (SQL: UPDATE ratings SET rating = ..., comment = ... WHERE user_id = ... AND recipe_id = ...), ja nē - tiek izveidots jauns ieraksts (SQL: INSERT INTO ratings (user_id, recipe_id, rating, comment) VALUES (...)). Pēc saglabāšanas tiek aprēķināts jaunais vidējais vērtējums receptei (SQL: SELECT AVG(rating) FROM ratings WHERE recipe_id = ...). Lietotājam tiek atgriezts saglabātais vērtējuma objekts un jaunais vidējais vērtējums.



8.att. Receptes vērtēšanas moduļa datu plūsmas diagramma

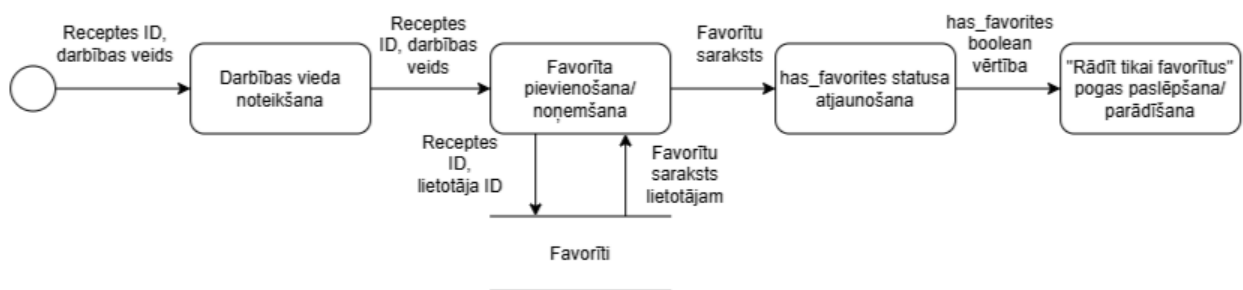
6. Favorītu pārvaldības modulis (skat. 9.att.).

Lietotājs norāda receptes ID un darbības veidu (pievienot vai noņemt). Pievienošanas gadījumā pārlūkprogramma nosūta pieprasījumu serverim (POST /favorites) ar receptes ID.

Serverī vispirms tiek pārbaudīts, vai šāds favorīts jau eksistē (SQL: SELECT EXISTS(SELECT 1 FROM favorites WHERE user_id = ... AND recipe_id = ...)), un, ja nē, tiek izveidots jauns ieraksts (SQL: INSERT INTO favorites (user_id, recipe_id) VALUES (...)).

Noņemšanas gadījumā pārlūkprogramma nosūta dzēšanas pieprasījumu (DELETE /favorites/{recipe_id}), un serverī ieraksts tiek dzēsts no datu bāzes (SQL: DELETE FROM favorites WHERE user_id = ... AND recipe_id = ...).

Abos gadījumos pēc darbības tiek pārbaudīts, vai lietotājam vēl ir kādi favorīti (SQL: SELECT EXISTS(SELECT 1 FROM favorites WHERE user_id = ...)), un atjaunināts has_favorites statuss. Lietotājam tiek atgriezts šis statuss, kas tiek izmantots, lai paslēptu vai parādītu pogu "Rādīt tikai favorītus".

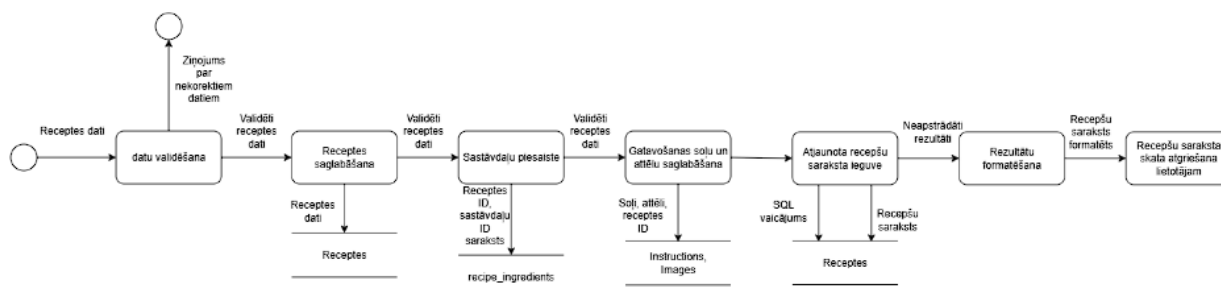


9.att. Favorītu pārvaldības moduļa datu plūsmas diagramma

7. Administratora receptes izveides modulis (skat. 10.att.).

Administrators ievada receptes pamatlaukus, sastāvdaļu sarakstu ar daudzumiem un vienībām, gatavošanas soļus un attēlus. Pārlūkprogramma nosūta pieprasījumu serverim (POST /admin/receptes) ar visiem datiem, ieskaitot failus. Tiek validēti visi lauki - nosaukums, apraksts, gatavošanas laiks, sarežģītības līmenis, ēdienreize un citi klasifikācijas lauki, kā arī sastāvdaļu, soļu un attēlu dati (maksimāli 5 attēli, katrs līdz 5 MB).

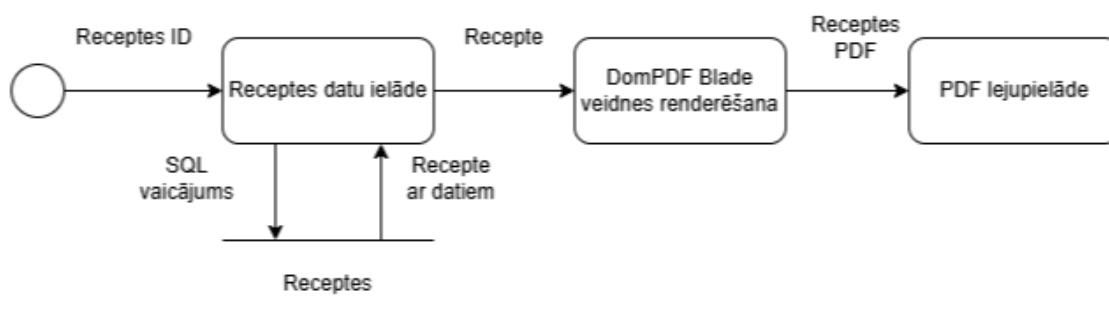
Pēc veiksmīgas validācijas datu bāzē tiek secīgi izpildītas vairākas saglabāšanas darbības: vispirms tiek izveidots receptes ieraksts (SQL: INSERT INTO recipes (name, description, cooking_time, ...) VALUES (...)); tad katrai sastāvdaļai tiek izveidots sasaistes ieraksts ar daudzumu un vienību (SQL: INSERT INTO recipe_ingredients (recipe_id, ingredient_id, quantity, unit_id) VALUES (...)); katrs gatavošanas solis tiek saglabāts ar automātiski piešķirtu soļa numuru (SQL: INSERT INTO instructions (recipe_id, step_number, description) VALUES (...)); attēli tiek pārvērsti base64 formātā un saglabāti ar faila informāciju (SQL: INSERT INTO images (recipe_id, data, mime_type, original_filename, file_size) VALUES (...)). Administrators tiek novirzīts atpakaļ uz receptes sarakstu ar statusa ziņojumu.



10.att. Administratora receptes izveides moduļa datu plūsmas diagramma

8. PDF dokumenta ģenerēšanas modulis (skat. 11.att.).

Lietotājs norāda receptes ID. Pārlūkprogramma nosūta pieprasījumu serverim (GET /api/recipes/{id}/pdf). Serverī tiek ielādēta recepte ar visiem saistītajiem datiem - gatavošanas soļiem, sastāvdaļām, attēlu, klasifikācijas laukiem - un aprēķināts vidējais vērtējums (SQL: SELECT recipes.*, (SELECT AVG(rating) FROM ratings WHERE recipe_id = recipes.id) AS average_rating FROM recipes WHERE id = ...). Vienību nosaukumi tiek iegūti no atsevišķas tabulas un saglabāti kešatmiņā uz 1 stundu (SQL: SELECT id, name FROM units). DomPDF bibliotēka pārveido HTML veidni par PDF dokumentu A4 formātā. Lietotājam tiek nosūtīts PDF fails kā lejupielāde ar nosaukumu recepte-{id}.pdf.



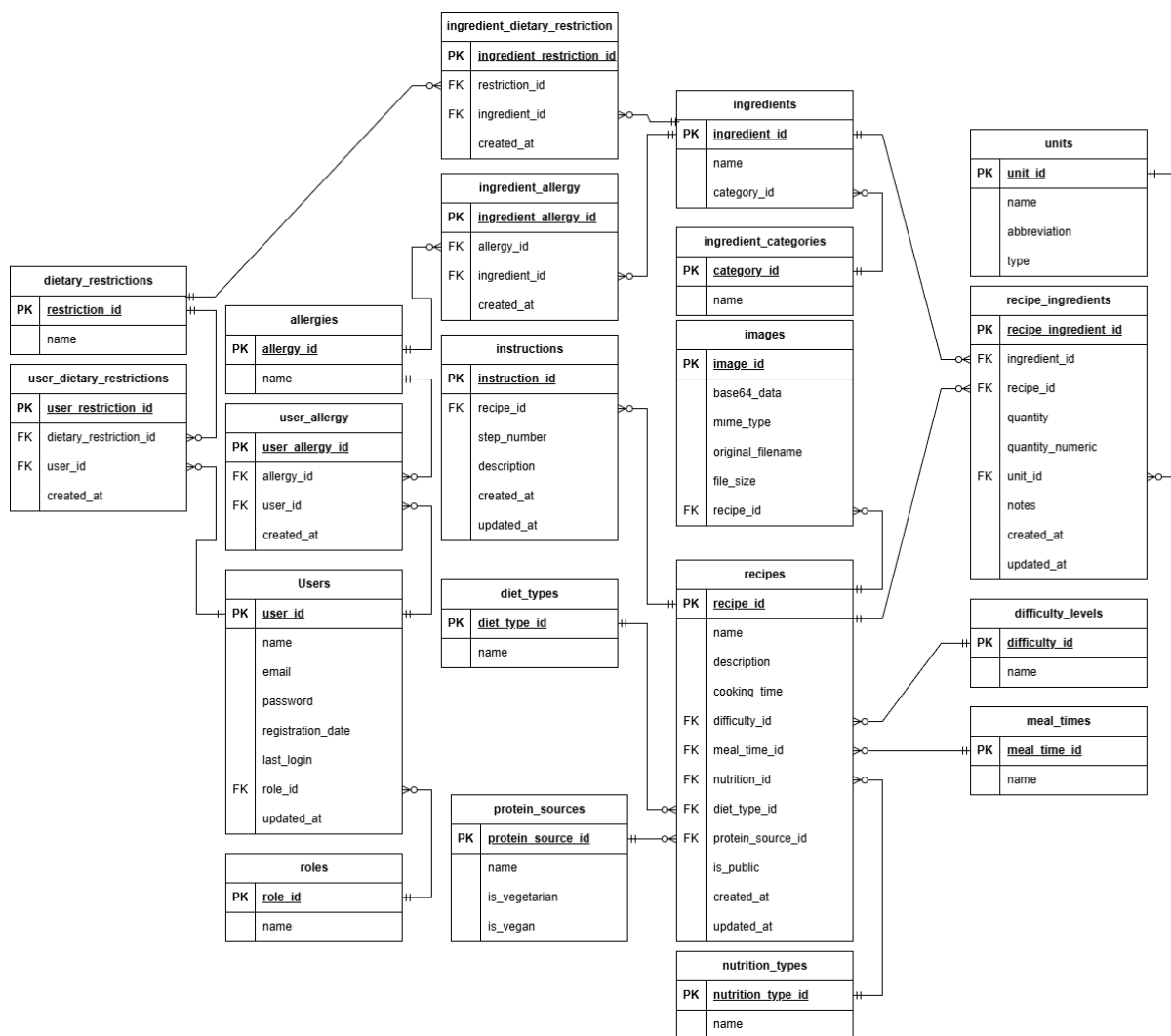
11.att. PDF dokumenta ģenerēšanas moduļa datu plūsmas diagramma

5. DATU STRUKTŪRU APRAKSTS

Datubāzes izveidošanai ir nepieciešama datubāzes fiziskā struktūra. Tā tika iegūta normalizējot ER diagrammu līdz trešajai normālformai (3NF), kuras rezultātā no entītijām izveidoja relācijas. Datubāze sastāv no 22 tabulām: 8 uzmeklēšanas tabulām (lookup tables), 5 galvenajām tabulām un 9 starptabulām (junction tables). Shēma satur informāciju par lietotājiem, lomām, receptēm, sastāvdaļām, gatavošanas instrukcijām, attēliem, alerģijām un uztura ierobežojumiem.

- Tabula “roles” glabā informāciju par lietotāju lomām.
- Tabula “difficulty_levels” glabā informāciju par receptes sarežģītības līmeņiem.
- Tabula “meal_times” glabā informāciju par ēdienreīžu tipiem.
- Tabula “nutrition_types” glabā informāciju par uztura mērķu kategorijām.
- Tabula “diet_types” glabā informāciju par diētu klasifikācijām.
- Tabula “protein_sources” glabā informāciju par olbaltumvielu avotiem.
- Tabula “ingredient_categories” glabā informāciju par sastāvdaļu kategorijām.
- Tabula “units” glabā informāciju par mērvienībām.
- Tabula “allergies” glabā informāciju par alerģiju veidiem.
- Tabula “dietary_restrictions” glabā informāciju par uztura ierobežojumiem.
- Tabula “users” glabā informāciju par reģistrētiem lietotājiem.
- Tabula “recipes” glabā informāciju par ēdienu receptēm.
- Tabula “images” glabā informāciju par recepšu attēliem.
- Tabula “ingredients” glabā informāciju par pieejamām sastāvdaļām.
- Tabula “instructions” glabā informāciju par receptes gatavošanas soļiem.
- Tabula “recipe_ingredients” glabā informāciju par receptēm un to sastāvdaļām.
- Tabula “user_allergy” glabā informāciju par lietotāju alerģijām.
- Tabula “user_dietary_restrictions” glabā informāciju par lietotāju uztura ierobežojumiem.
- Tabula “ingredient_allergy” glabā informāciju par sastāvdaļu un alerģiju saistību.
- Tabula “ingredient_dietary_restriction” glabā informāciju par sastāvdaļu un uztura ierobežojumu saistību.
- Tabula “favorites” glabā informāciju par lietotāju saglabātajām favorītu receptēm.
- Tabula “ratings” glabā informāciju par lietotāju vērtējumiem receptēm.

Tabulu saišu shēmu skatīt 12. attēlā.



12.att. Datubāzes tabulu saišu shēma

Tabula “roles” satur informāciju par lietotāju lomām sistēmā. Tabula ir savienota ar tabulu “users” ar saiti viens-pret-daudziem, jo vienam lietotājam ir tikai viena loma, bet tā pati loma var būt vairākiem lietotājiem (skat. 5.1. tabulu).

5.1.tabula

Tabulas “roles” struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	role_id	TINYINT UNSIGNED	-	Lomas unikālais identifikators (PK)
2	name	VARCHAR	50	Lomas nosaukums (unikāls)

Tabula “difficulty_levels” ir uzmeklēšanas tabula, kas satur receptes sarežģītības līmeņus (Viegli, Vidēji, Grūti). Tabula ir savienota ar tabulu “recipes” ar saiti viens-pret-daudziem (skat. 5.2. tabulu).

5.2.tabula

Tabulas “difficulty_levels” struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	difficulty_id	TINYINT UNSIGNED	-	Sarežģītības līmeņa unikālais identifikators (PK)
2	name	VARCHAR	50	Sarežģītības līmeņa nosaukums (unikāls)

Tabula “meal_times” ir uzmeklēšanas tabula, kas satur ēdienrežu tipus (Brokastis, Pusdienas, Vakariņas, Uzkodas, Deserts). Tabula ir savienota ar tabulu “recipes” ar saiti viens-pret-daudziem (skat. 5.3. tabulu).

5.3.tabula

Tabulas “meal_times” struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	meal_time_id	TINYINT UNSIGNED	-	Ēdienreizes unikālais identifikators (PK)
2	name	VARCHAR	50	Ēdienreizes nosaukums (unikāls)

Tabula “nutrition_types” ir uzmeklēšanas tabula, kas satur uztura mērķu kategorijas (Augsts olbaltumvielu saturs, Maz ogļhidrātu, Maz tauku, Balancēts uzturs, Daudz šķiedrvielu). Tabula ir savienota ar tabulu “recipes” ar saiti viens-pret-daudziem (skat. 5.4. tabulu).

5.4.tabula

Tabulas “nutrition_types” struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	nutrition_type_id	TINYINT UNSIGNED	-	Uztura tipa unikālais identifikators (PK)
2	name	VARCHAR	100	Uztura tipa nosaukums (unikāls)

Tabula “diet_types” ir uzmeklēšanas tabula, kas satur diētu klasifikācijas (Veģitāra, Vegāna, Keto, Paleo). Tabula ir savienota ar tabulu “recipes” ar saiti viens-pret-daudziem (skat. 5.5. tabulu).

5.5.tabula

Tabulas “diet_types” struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	diet_type_id	TINYINT UNSIGNED	-	Diētas tipa unikālais identifikators (PK)

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
2	name	VARCHAR	100	Diētas tipa nosaukums (unikāls)

Tabula “protein_sources” ir uzmeklēšanas tabula, kas satur olbaltumvielu avotu opcijas (Vista, Liellops, Tofu, Lēcas u.c.). Tabula ir savienota ar tabulu “recipes” ar saiti viens-pret-daudziem (skat. 5.6. tabulu).

5.6.tabula

Tabulas “protein_sources” struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	protein_source_id	SMALLINT UNSIGNED	-	Olbaltumvielu avota unikālais identifikators (PK)
2	name	VARCHAR	100	Olbaltumvielu avota nosaukums (unikāls)
3	is_vegetarian	TINYINT(1)	-	Vai piemērots veģetāriešiem (noklusējums 0)
4	is_vegan	TINYINT(1)	-	Vai piemērots vegāniem (noklusējums 0)

Tabula “ingredient_categories” ir uzmeklēšanas tabula, kas satur sastāvdaļu kategorijas (Dārzeņi, Augļi, Piena produkti, Gaļa u.c.). Tabula ir savienota ar tabulu “ingredients” ar saiti viens-pret-daudziem (skat. 5.7. tabulu).

5.7.tabula

Tabulas “ingredient_categories” struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	category_id	SMALLINT UNSIGNED	-	Kategorijas unikālais identifikators (PK)
2	name	VARCHAR	100	Kategorijas nosaukums (unikāls)

Tabula “units” ir uzmeklēšanas tabula, kas satur mērvienības receptēs (grami, ml, tējkarotes u.c.). Tabula ir savienota ar tabulu “recipe_ingredients” ar saiti viens-pret-daudziem (skat. 5.8. tabulu).

5.8.tabula

Tabulas “units” struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	unit_id	SMALLINT UNSIGNED	-	Mērvienības unikālais identifikators (PK)
2	name	VARCHAR	30	Mērvienības pilnais nosaukums (unikāls)
3	abbreviation	VARCHAR	10	Mērvienības saīsinājums (unikāls)

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
4	type	ENUM	-	Mērvienības tips (volume, weight, count, other)

Tabula “allergies” satur informāciju par alerģiju veidiem. Tabula ir savienota ar tabulu “users” ar saiti daudzi-pret-daudziem caur starptabulu “user_allergy” un ar tabulu “ingredients” ar saiti daudzi-pret-daudziem caur starptabulu “ingredient_allergy” (skat. 5.9. tabulu).

5.9.tabula

Tabulas “allergies” struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	allergy_id	BIGINT UNSIGNED	-	Alerģijas unikālais identifikators (PK)
2	name	VARCHAR	255	Alerģijas nosaukums (unikāls)

Tabula “dietary_restrictions” satur informāciju par uztura ierobežojumiem (Bezglutēna, Veģetāra, Zema ogļhidrātu u.c.). Tabula ir savienota ar tabulu “users” ar saiti daudzi-pret-daudziem caur starptabulu “user_dietary_restrictions” un ar tabulu “ingredients” ar saiti daudzi-pret-daudziem caur starptabulu “ingredient_dietary_restriction” (skat. 5.10. tabulu).

5.10.tabula

Tabulas “dietary_restrictions” struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	restriction_id	BIGINT UNSIGNED	-	Uztura ierobežojuma unikālais identifikators (PK)
2	name	VARCHAR	255	Uztura ierobežojuma nosaukums (unikāls)

Tabula “users” satur reģistrētu lietotāju datus. Tabula ir savienota ar tabulu “roles” ar saiti daudzi-pret-vienu caur lauku role_id, ar tabulām “allergies” un “dietary_restrictions” ar saiti daudzi-pret-daudziem caur starptabulām (skat. 5.11. tabulu).

5.11.tabula

Tabulas “users” struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	user_id	BIGINT UNSIGNED	-	Lietotāja unikālais identifikators (PK)
2	name	VARCHAR	255	Lietotāja vārds
3	email	VARCHAR	255	Lietotāja e-pasta adrese (unikāla)
4	password	VARCHAR	255	Lietotāja šifrētā parole
5	role_id	TINYINT UNSIGNED	-	Lomas identifikators (FK -> roles.role_id)
6	registration_date	DATE	-	Lietotāja reģistrācijas datums

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
7	last_login	DATETIME	-	Pēdējās pieteikšanās datums un laiks
8	updated_at	TIMESTAMP	-	Ieraksta atjaunināšanas laiks

Tabula “recipes” satur informāciju par sistēmā pieejamām receptēm. Tabula ir savienota ar uzmeklēšanas tabulām “difficulty_levels”, “meal_times”, “nutrition_types”, “diet_types” un “protein_sources” ar saiti daudzi-pret-vienu. Tā ir savienota ar tabulām “images” un “instructions” ar saiti viens-pret-daudzām, kā arī ar tabulu “ingredients” ar saiti daudzi-pret-daudzām caur starptabulu “recipe_ingredients” (skat. 5.12. tabulu).

5.12.tabula

Tabulas “recipes” struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	recipe_id	BIGINT UNSIGNED	-	Receptes unikālais identifikators (PK)
2	name	VARCHAR	255	Receptes nosaukums
3	description	TEXT	-	Receptes detalizēts apraksts
4	cooking_time	SMALLINT UNSIGNED	-	Gatavošanas laiks minūtēs
5	difficulty_id	TINYINT UNSIGNED	-	Sarežģītības līmeņa ID (FK -> difficulty_levels.difficulty_id)
6	meal_time_id	TINYINT UNSIGNED	-	Ēdienreizes ID (FK -> meal_times.meal_time_id)
7	nutrition_id	TINYINT UNSIGNED	-	Uztura tipa ID (FK -> nutrition_types.nutrition_type_id)
8	diet_type_id	TINYINT UNSIGNED	-	Diētas tipa ID (FK -> diet_types.diet_type_id)
9	protein_source_id	SMALLINT UNSIGNED	-	Olbaltumvielu avota ID (FK -> protein_sources.protein_source_id)
10	is_public	TINYINT(1)	-	Publiskuma statuss (1=publiska)
11	created_at	TIMESTAMP	-	Ieraksta izveidošanas laiks
12	updated_at	TIMESTAMP	-	Ieraksta atjaunināšanas laiks

Tabula “images” satur informāciju par recepšu attēliem Base64 formātā. Tabula ir savienota ar tabulu “recipes” ar saiti daudzi-pret-vienu caur lauku recipe_id, kas nozīmē, ka vienai receptei var būt vairāki attēli (skat. 5.13. tabulu).

5.13.tabula

Tabulas “images” struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	image_id	BIGINT UNSIGNED	-	Attēla unikālais identifikators (PK)
2	recipe_id	BIGINT UNSIGNED	-	Receptes identifikators (FK -> recipes.recipe_id)
3	base64_data	LONGTEXT	-	Base64 kodēts attēla saturs
4	mime_type	VARCHAR	50	Attēla MIME tips (piem., image/png)
5	original_filename	VARCHAR	255	Orģinālā faila nosaukums
6	file_size	INT UNSIGNED	-	Faila izmērs baitos

Tabula “ingredients” satur informāciju par sistēmā pieejamām sastāvdaļām. Tabula ir savienota ar tabulu “ingredient_categories” ar saiti daudzi-pret-vienu, ar tabulu “recipes” ar saiti daudzi-pret-daudziem caur starptabulu “recipe_ingredients”, kā arī ar tabulām “allergies” un “dietary_restrictions” ar saiti daudzi-pret-daudziem (skat. 5.14. tabulu).

5.14.tabula

Tabulas “ingredients” struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	ingredient_id	BIGINT UNSIGNED	-	Sastāvdaļas unikālais identifikators (PK)
2	name	VARCHAR	255	Sastāvdaļas nosaukums
3	category_id	SMALLINT UNSIGNED	-	Kategorijas ID (FK -> ingredient_categories.category_id)

Tabula “instructions” satur informāciju par receptes gatavošanas soļiem. Tabula ir savienota ar tabulu “recipes” ar saiti daudzi-pret-vienu caur lauku recipe_id. Kombinācija recipe_id un step_number ir unikāla (skat. 5.15. tabulu).

5.15.tabula

Tabulas “instructions” struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	instruction_id	BIGINT UNSIGNED	-	Soļa unikālais identifikators (PK)
2	recipe_id	BIGINT UNSIGNED	-	Receptes identifikators (FK -> recipes.recipe_id)
3	step_number	SMALLINT UNSIGNED	-	Soļa kārtas numurs
4	description	TEXT	-	Soļa detalizēts apraksts
5	created_at	TIMESTAMP	-	Ieraksta izveidošanas laiks
6	updated_at	TIMESTAMP	-	Ieraksta atjaunināšanas laiks

Tabula “recipe_ingredients” ir starptabula, kas realizē daudzi-pret-daudziem saiti starp tabulām “recipes” un “ingredients”. Tā satur informāciju par katras receptes sastāvdaļām un to daudzumiem. Kombinācija recipe_id un ingredient_id ir unikāla (skat. 5.16. tabulu).

5.16.tabula

Tabulas “recipe_ingredients” struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	recipe_ingredient_id	BIGINT UNSIGNED	-	Ieraksta unikālais identifikators (PK)
2	recipe_id	BIGINT UNSIGNED	-	Receptes identifikators (FK -> recipes.recipe_id)
3	ingredient_id	BIGINT UNSIGNED	-	Sastāvdaļas identifikators (FK -> ingredients.ingredient_id)
4	quantity	VARCHAR	255	Daudzums teksta formātā
5	quantity_numeric	DECIMAL(8, 2)	-	Daudzums skaitliskā formātā
6	unit_id	SMALLINT UNSIGNED	-	Mērvienības ID (FK -> units.unit_id)
7	notes	TEXT	-	Papildu piezīmes
8	created_at	TIMESTAMP	-	Ieraksta izveidošanas laiks
9	updated_at	TIMESTAMP	-	Ieraksta atjaunināšanas laiks

Tabula “user_allergy” ir starptabula, kas realizē daudzi-pret-daudziem saiti starp tabulām “users” un “allergies”, ļaujot lietotājiem norādīt savas alerģijas. Kombinācija user_id un allergy_id ir unikāla (skat. 5.17. tabulu).

5.17.tabula

Tabulas “user_allergy” struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	user_allergy_id	BIGINT UNSIGNED	-	Ieraksta unikālais identifikators (PK)
2	user_id	BIGINT UNSIGNED	-	Lietotāja identifikators (FK -> users.user_id)
3	allergy_id	BIGINT UNSIGNED	-	Alerģijas identifikators (FK -> allergies.allergy_id)
4	created_at	TIMESTAMP	-	Ieraksta izveidošanas laiks

Tabula “user_dietary_restrictions” ir starptabula, kas realizē daudzi-pret-daudziem saiti starp tabulām “users” un “dietary_restrictions”, ļaujot lietotājiem norādīt savus uztura ierobežojumus. Kombinācija user_id un dietary_restriction_id ir unikāla (skat. 5.18. tabulu).

5.18.tabula

Tabulas “user_dietary_restrictions” struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	user_restriction_id	BIGINT UNSIGNED	-	Ieraksta unikālais identifikators (PK)
2	user_id	BIGINT UNSIGNED	-	Lietotāja identifikators (FK -> users.user_id)
3	dietary_restriction_id	BIGINT UNSIGNED	-	Uztura ierobežojuma ID (FK -> dietary_restrictions.restriction_id)
4	created_at	TIMESTAM P	-	Ieraksta izveidošanas laiks

Tabula "ingredient_allergy" ir starptabula, kas realizē daudzi-pret-daudziem saiti starp tabulām "ingredients" un "allergies", norādot kuras sastāvdaļas izraisa kādas alerģijas. Kombinācija ingredient_id un allergy_id ir unikāla (skat. 5.19. tabulu).

5.19.tabula

Tabulas "ingredient_allergy" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	ingredient_allergy_id	BIGINT UNSIGNED	-	Ieraksta unikālais identifikators (PK)
2	ingredient_id	BIGINT UNSIGNED	-	Sastāvdaļas identifikators (FK -> ingredients.ingredient_id)
3	allergy_id	BIGINT UNSIGNED	-	Alerģijas identifikators (FK -> allergies.allergy_id)
4	created_at	TIMESTAM P	-	Ieraksta izveidošanas laiks

5.20.tabula

Tabulas "ingredient_dietary_restriction" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	ingredient_restriction_id	BIGINT UNSIGNED	-	Ieraksta unikālais identifikators (PK)
2	ingredient_id	BIGINT UNSIGNED	-	Sastāvdaļas identifikators (FK -> ingredients.ingredient_id)
3	restriction_id	BIGINT UNSIGNED	-	Uztura ierobežojuma ID (FK -> dietary_restrictions.restriction_id)
4	created_at	TIMESTAM P	-	Ieraksta izveidošanas laiks

Tabula "favorites" ir starptabula, kas realizē daudzi-pret-daudziem saiti starp tabulām "users" un "recipes", ļaujot lietotājiem saglabāt receptes savos favorītos. Kombinācija user_id un recipe_id ir unikāla (skat. 5.21. tabulu).

5.21.tabula

Tabulas "favorites" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	favorite_id	BIGINT UNSIGNED	-	Ieraksta unikālais identifikators (PK)
2	user_id	BIGINT UNSIGNED	-	Lietotāja identifikators (FK -> users.user_id)
3	recipe_id	BIGINT UNSIGNED	-	Receptes identifikators (FK -> recipes.recipe_id)
4	created_at	TIMESTAM P	-	Ieraksta izveidošanas laiks
5	updated_at	TIMESTAM P		Ieraksta atjaunināšanas laiks

Tabula "ratings" glabā lietotāju vērtējumus receptēm. Tabula ir savienota ar tabulu "users" ar saiti daudzi-pret-vienu un ar tabulu "recipes" ar saiti daudzi-pret-vienu. Kombinācija user_id un recipe_id ir unikāla, kas nozīmē, ka viens lietotājs var novērtēt vienu recepti tikai vienu reizi (skat. 5.22. tabulu).

5.22.tabula

Tabulas "ratings" struktūra

Nr	Lauka nosaukums	Tips	Izmērs	Apraksts
1	rating_id	BIGINT UNSIGNED	-	Ieraksta unikālais identifikators (PK)
2	user_id	BIGINT UNSIGNED	-	Lietotāja identifikators (FK -> users.user_id)
3	recipe_id	BIGINT UNSIGNED	-	Receptes identifikators (FK -> recipes.recipe_id)
4	rating	TINYINT UNSIGNED		Vērtējums skalā no 1 līdz 5
5	comment	TEXT		Neobligāts komentārs par recepti
6	created_at	TIMESTAM P	-	Ieraksta izveidošanas laiks
7	updated_at	TIMESTAM P		Ieraksta atjaunināšanas laiks

6. LIETOTĀJA CEĻVEDIS

6.1. Sistēmas prasības aparatūrai un programmatūrai

Ēdienu recepšu meklēšanas un ģenerēšanas sistēma ir tīmekļa lietotne, kas ir piemērota ērtai lietošanai lielākajā daļā ierīču, ja tās atbilst sekojošām prasībām:

- Operētājsistēma:
 - Windows 10 vai jaunāka versija;
 - macOS 12 vai jaunāka versija;
 - Linux (Ubuntu 20.04+).
- Interneta pārlūks:
 - Google Chrome 105 vai jaunāka versija;
 - Microsoft Edge 105 vai jaunāka versija;
 - Mozilla FireFox 121 vai jaunāka versija;
 - Safari 15.4 vai jaunāka versija;
 - Opera 91 vai jaunāka versija.
- Aparatūra:
 - RAM minimums 2 GB;
 - Diska vieta minimums 500 MB;
 - Interneta savienojums (ārējiem pakalpojumiem: DeepSeek API, Google OAuth, ...).

6.2. Sistēmas instalācija un palašana

Sistēma pieejama tiešsaistē - <https://foodyml-aleksisdp41.rvtdev.tech>.

Lai instalētu un palaistu sistēmu lokāli, lietotājam vispirms ir jāparūpējās par sekojoša programmnodrošinājuma instalēšanu uz izmantojamās ierīces:

- Laragon (iekļauj uzreiz PHP, MySQL un Apache vienā instalācijā);
- Composer;
- Node.js;
- Git;
- Visual Studio Code.

Nepieciešami arī ārējo pakalpojumu konti. Pirms konfigurācijas nepieciešami šādi konti/atslēgas:

- DeepSeek API (AI recepšu ģenerēšana):
 - Reģistrējies platform.deepseek.com;
 - Dodies uz API Keys un izveido jaunu atslēgu;
 - Nokopē un saglabā atslēgu.
- Google OAuth (pierakstīšanās ar Google):
 - Atver console.cloud.google.com;
 - Izveido jaunu projektu;
 - APIs & Services -> Credentials -> Create Credentials -> OAuth 2.0 Client ID;
 - Application type: Web application;
 - Authorized redirect URI: <http://localhost:8000/auth/google/callback>;
 - Nokopē/saglabā Client ID un Client Secret.

- Gmail SMTP (e-pasta izsūtīšana):
 - Google kots -> Drošība -> Divpakāpju verifikācija (jābūt ieslēgtai);
 - Drošība -> App passwords -> izveido jaunu paroli lietotnei;
 - Nokopē/saglabā 16 simbolu paroli.

1. Projekta klonēšana no GitHub:

1.1. Atveriet termināli un izpildiet komandu (skat. 13. att. Git Clone);

```
git clone https://github.com/22DP1AKaha/RecipeGenerator.git
```

13.att. Git Clone

1.2. Ieejiet projekta mapē (skat. 14. att. Projekta mape);

```
cd RecipeGenerator
```

14.att. Projekta mape

2. Atkarību uzstādīšana:

2.1. PHP atkarību uzstādīšana. Terminālī izpildiet komandu (skat. 15. att. PHP atkarības);

```
composer install
```

15.att. PHP atkarības

2.2. JavaScript atkarību uzstādīšana. Terminālī izpildiet komandu (skat. 16. att. JS atkarības);

```
npm install
```

16.att. JS atkarības

3. Vides konfigurēšana:

3.1. Nokopē konfigurācijas failu (skat. 17. att. Konfigurācijas fails);

```
npm install
```

17.att. Konfigurācijas fails

3.2. Atver .env failu un aizpildi šādus laukus (skat. 18. att. .env fails);

```

# Datubāze
DB_DATABASE=recipegenerator
DB_USERNAME=root
DB_PASSWORD=          # tava MySQL parole

# E-pasts (Gmail)
MAIL_MAILER=smtp
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=tavs@gmail.com
MAIL_PASSWORD=xxxx xxxx xxxx xxxx    # App password
MAIL_FROM_ADDRESS=tavs@gmail.com
MAIL_FROM_NAME="FOODYML"

# DeepSeek AI
DEEPSEEK_API_KEY=sk-...
DEEPSEEK_TIMEOUT=60

# Google OAuth
GOOGLE_CLIENT_ID=...
GOOGLE_CLIENT_SECRET=...
GOOGLE_REDIRECT_URI=http://localhost:8000/auth/google/callback

```

18.att. .env fails

4. Datubāzes izveide:

4.1. Ieej Laravel un izpildi skriptu (skat. 19. att. DB izveide);

```
CREATE DATABASE recipegenerator CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

19.att. DB izveide

4.2. Tabulu izveidei un aizpildei ar testa datiem atgriezies uz termināli un izpildi komandu (skat. 20. att. DB aizpildīšana);

```
php artisan migrate --seed
```

20.att. DB aizpildīšana

5. Projekta palaišana:

5.1. Nepieciešami divi atsevišķi termināļi, kuros ir ieiets projekta mapē:

5.1.1. Pirmajā terminālī izpildi komandu (skat. 21. att. PHP serveris);

```
php artisan serve
```

21.att. PHP serveris

5.1.2. Otrajā terminālī izpildi komandu (skat. 22. att. Frontend palaišana);

```
npm run dev
```

6. Administratora konta izveide:
 - 6.1. Reģistrējaties FOODYML sistēmā;
 - 6.2. Laragon izpildiet skriptu, aizvietojot ē-pastu ar to, ko izmantojāt reģistrācijā (skat. 23. att. Admin izveide);

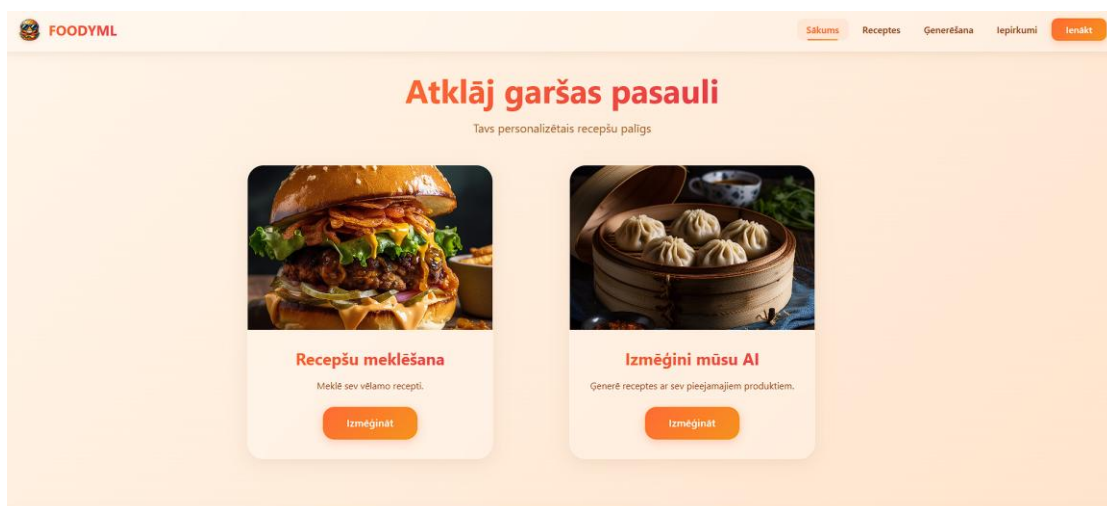
```
UPDATE users
SET role_id = (SELECT id FROM roles WHERE name = 'Administrators')
WHERE email = 'tavs@example.com';
```

23.att. Admin izveide

6.3. Programmas apraksts

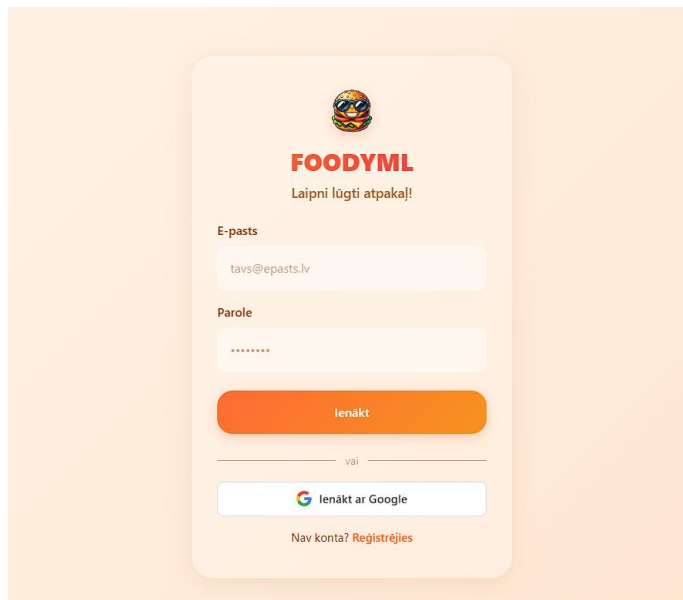
Autentifikācija.

Atverot tīmekļa lietotni pārlūkprogrammā, lietotājam tiek parādīta sākuma lapa (skat. 24. att.).



24.att. Sākuma lapa

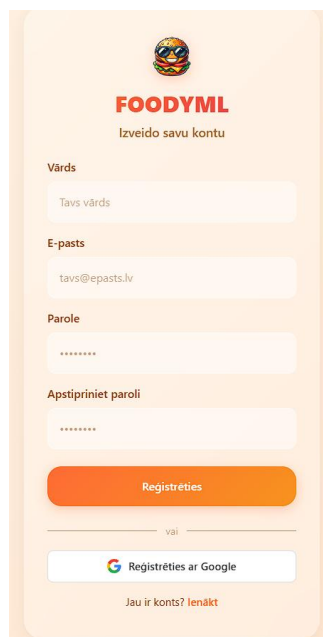
Nospiežot pogu kreisajā augšējā stūrī (“Ienākt”), lietotājs tiks pārvirzīts uz autentifikācijas formu (skat. 25. att.).



The login form for FOODYML features a hamburger icon at the top. Below it, the text "FOODYML" is displayed in red, followed by the greeting "Laipni lūgti atpakaļ!". The form includes an "E-pasts" field with the placeholder "tavs@epasts.lv", a "Parole" field with a masked password "*****", and an orange "Ienākt" button. A horizontal separator with "vai" in the center is followed by a white button with the Google logo and the text "Ienākt ar Google". At the bottom, a link "Nav konta? Reģistrējies" is provided.

25.att. Autentifikācijas forma

Šajā lapā ir iespēja ienākt ar sistēmā esošu kontu vai ienākt ar Google kontu, bet ja nav vēl izveidots konts, nospiežot pogu “Reģistrējies”, lietotājs tiks pārvirzīts uz reģistrācijas formu (skat. 26. att.)



The registration form for FOODYML features a hamburger icon at the top. Below it, the text "FOODYML" is displayed in red, followed by the instruction "Izveido savu kontu". The form includes a "Vārds" field with the placeholder "Tavs vārds", an "E-pasts" field with the placeholder "tavs@epasts.lv", a "Parole" field with a masked password "*****", and a confirmation field labeled "Apstipriniet paroli" with a masked password "*****". An orange "Reģistrēties" button is positioned below these fields. A horizontal separator with "vai" in the center is followed by a white button with the Google logo and the text "Reģistrēties ar Google". At the bottom, a link "Jau ir kants? Ienākt" is provided.

26.att. Reģistrācijas forma

Pēc pareizu datu ievades lietotājs tiks pārvirzīts atpakaļ uz sākuma lapu.

Recepšu funkcionalitāte

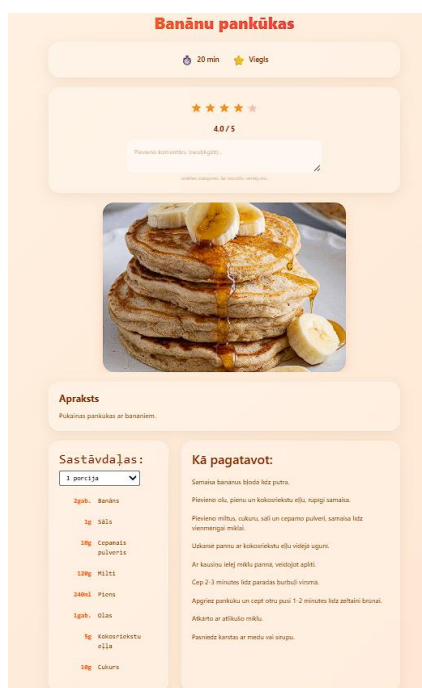
No sākuma lapas pārvirzoties uz lapu “Receptes”, atvērsies galvenā recepšu meklēšanas lapa, kur lietotājam ir iespēja meklēt, filtrēt un kārtot receptes (skat. 27. att.).



27.att. Recepšu meklētājs

Uzbraucot ar datorpeli uz kādas no receptēm, parādīsies poga “Pievienot favorītiem” (sirsniņas ikona) un receptes vidējais vērtējums.

Nospiežot uz kādas no receptēm, tiks atvērta receptes detalizēta lapa, kur ir informācija par receptes sarežģītības līmeni, pagatavošanas ilgums, apraksts, nepieciešamajām sastāvdaļām un gatavošanas soli, kā arī ir iespēja novērtēt recepti un pievienot tai komentāru (skat. 28. att).



28.att. Detalizēta receptes informācija

Receptes detalizētajā lapā ir arī poga “Lejupielādēt PDF”, kas saglabās doto recepti PDF dokumentā (skat. 29. att).



29.att. Lejupielādēt PDF

Receptes PDF tiek ģenerēts melnbalts, lai to būtu ērti izdrukāt (skat. 30. att.).

FOODYML

Banānu pankūkas

★ ★ ★ ★ ☆ 4.0 / 5.0

Gatavošanas laiks: 20 min

Ēdienreize: Brokastis

Diētas tips: Veģetāra

Grūtības pakāpe: Viegls

Uzturs: Veģetāriešiem

Olbaltumvielu avots: Olas

Apraksts

Pūkainas pankūkas ar banāniem.

Sastāvdaļas

2.00 gab.

Banāns

1.00 g

Sāls

10.00 g

Cepamais pulveris

120.00 g

Milti

30.att. Receptes PDF

Recepšu ģenerēšana

Pārvirzoties uz lapu “Ģenerēšana”, tiks atvērta lapa, kur ir iespējams izvēlēties sastāvdaļas un ģenerēt jaunu recepti izmantojot DeepSeek API (skat. 31. att.).



31.att. Recepšu ģenerators

Atverot aizvērtos sarakstus, tiks parādītas sastāvdaļas, kas atbilst dotajai kategorijai. Izvēloties vismaz 3 jebkuras sastāvdaļas parādīsies arī poga “Ģenerēt recepti” (skat. 32. att.).



32.att. Recepšu ģenerēšana

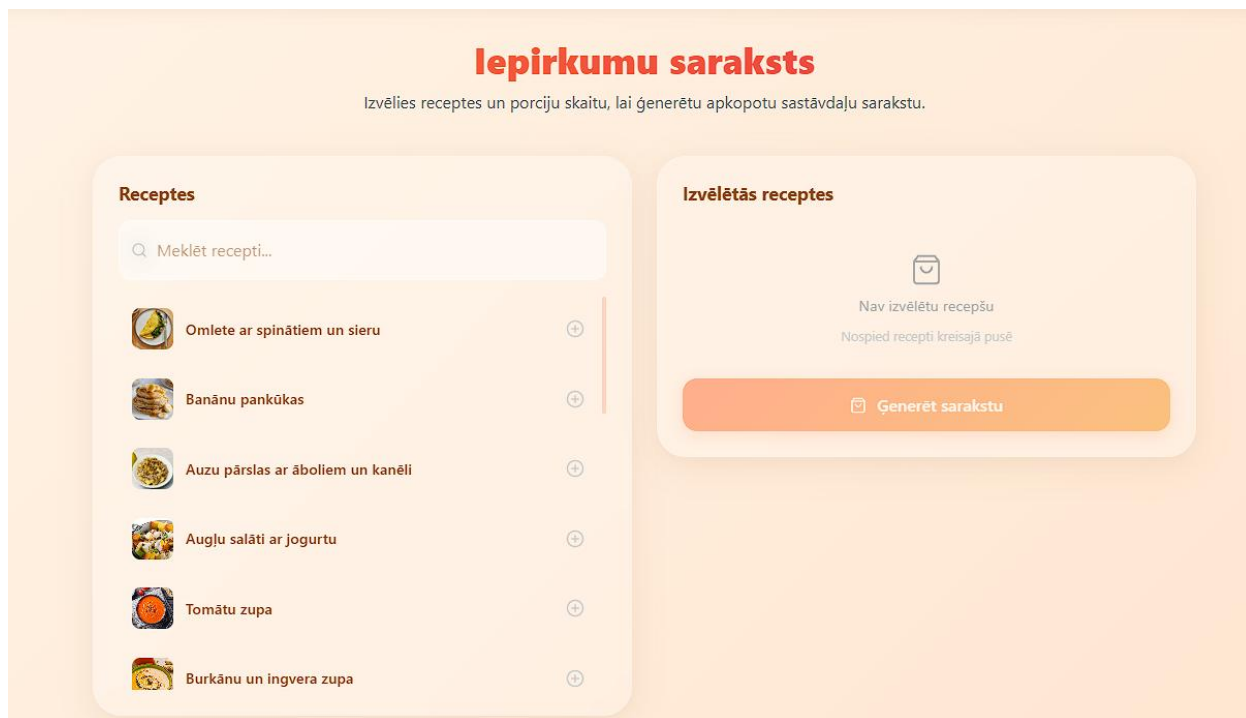
Nospiežot pogu “Ģenerēt recepti”, lietotājs tiks pārvirzīts uz lapas apakšu, kur tiks parādīta ģenerētā recepte no iepriekš norādītajām sastāvdaļām (skat. 33. att.).



33.att. Ģenerētā recepte

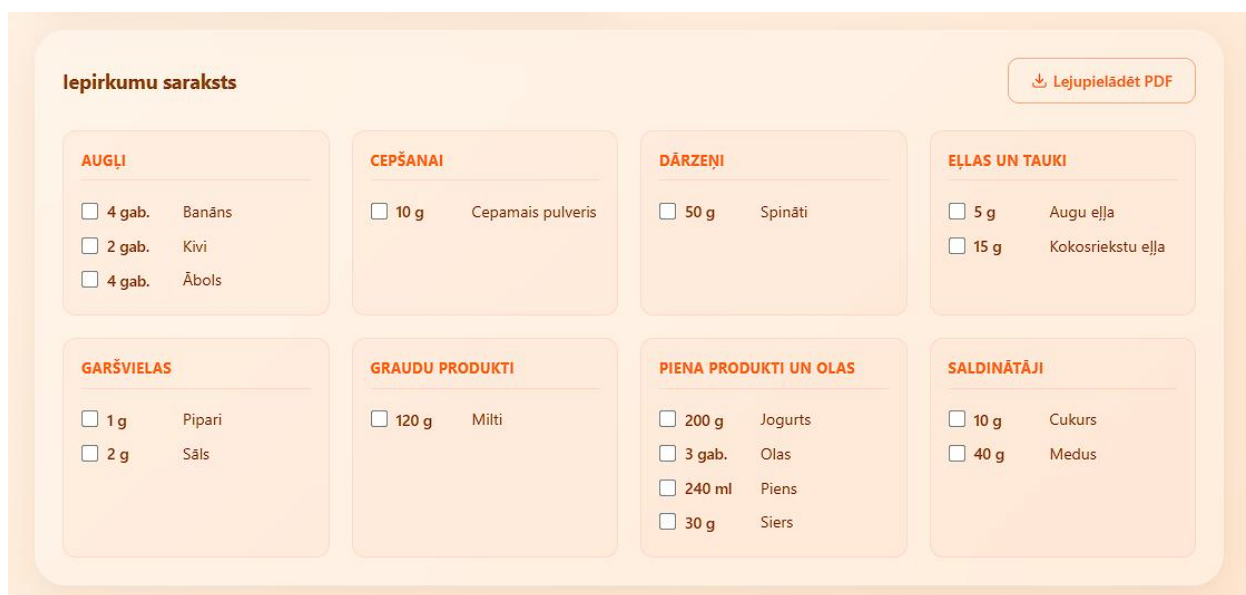
Sastāvdaļu saraksta veidošana

Pārvirzoties uz sadaļu “Iepirkumi”, tiks atvērta lapa, kur ir iespējams izvēlēties receptes un porciju skaitu, lai iegūtu visu sastāvdaļu sarakstu, kas nepieciešams, lai pagatavotu izvēlētajā receptē (skat. 34. att.).



34.att. Iepirkumu saraksts

Izvēloties vēlamās receptes un porciju skaitu, parādīsies poga “Ģenerēt sarakstu”, pēc tās nospiešanas lietotājs iegūs apkopotu sastāvdaļu sarakstu (skat. 35. att.).



35.att. Ģenerēts sastāvdaļu saraksts

Lietotājam ir iespēja arī lejupielādēt saraksta PDF (skat. 36. att.).

FOODYML

Iepirkumu saraksts

Ģenerēts: 19.04.2026 11:52

Iekļautās receptes:

Omlete ar spinātiem un sieru **(1x)** Augļu salāti ar jogurtu **(2x)** Banānu pankūkas **(1x)**

AUGĻI

☐ **4 gab.** Banāns

☐ **2 gab.** Kivi

☐ **4 gab.** Ābols

CEPŠANAI

☐ **10 g** Cepamais pulveris

DĀRZENĪ

☐ **50 g** Spināti

EĻLAS UN TAUKI

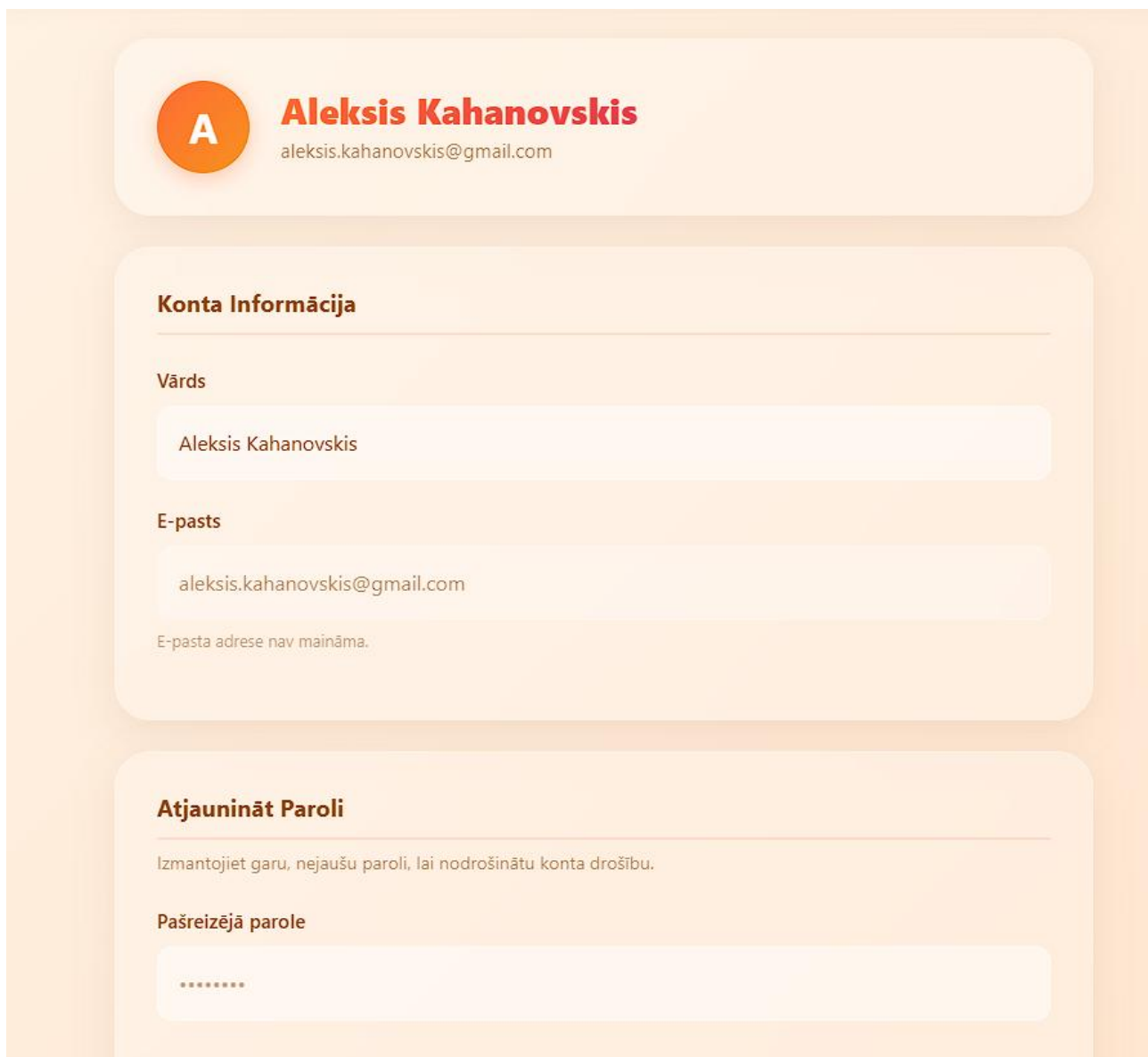
☐ **5 g** Augu eļļa

☐ **15 g** Kokosriekstu eļļa

36.att. Sastāvdaļu saraksta PDF

Profila funkcijas

Autentificējušam lietotājam ir sadaļa “Profils”, pārvirzoties uz šo sadaļu, tiks atvērta lapa ar lietotāja informāciju (vārds, uzvārds, e-pasts utt.) (skat. 37. att.)



The image shows a user profile page with a light orange background. At the top, there is a profile card containing a circular orange avatar with a white letter 'A', the name 'Aleksis Kahanovskis' in bold red text, and the email 'aleksis.kahanovskis@gmail.com' in smaller black text. Below this, there are two main sections. The first section, titled 'Konta Informācija' in bold brown text, contains two input fields: 'Vārds' (Name) with the value 'Aleksis Kahanovskis' and 'E-pasts' (Email) with the value 'aleksis.kahanovskis@gmail.com'. Below the email field, there is a small grey note: 'E-pasta adrese nav maināma.' (Email address is not changeable). The second section, titled 'Atjaunināt Paroli' (Update Password) in bold brown text, includes a grey instruction: 'Izmantojiet garu, nejaušu paroli, lai nodrošinātu konta drošību.' (Use a long, random password to ensure account security). Below this is a label 'Pašreizējā parole' (Current password) followed by a password input field with a masked password '*****'.

37.att. Profila lapa

Lapas apakšā ir bloki “Alerģijas” un “Diētas Ierobežojumi”, kur lietotājs var norādīt savas alerģijas pret ēdieniem un diētas ierobežojumus, kas receptšu meklēšanā atļaus filtrēt pēc lietotājam piemērotām receptēm (skat. 38. att.).

The image shows a user interface with two sections for filtering recipes. The first section, titled "Diētas Ierobežojumi" (Dietary Restrictions), contains nine checkboxes arranged in a 3x3 grid. The second section, titled "Alerģijas" (Allergies), also contains nine checkboxes arranged in a 3x3 grid. All checkboxes are currently unchecked.

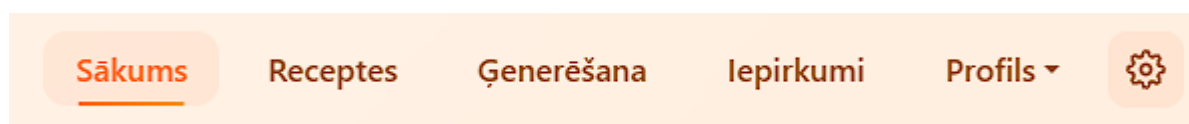
Diētas Ierobežojumi		
☐ Bezglutēna	☐ Veģetāra	☐ Vegāna
☐ Zema ogļhidrātu	☐ Zema tauku saturs	☐ Bez piena produktiem
☐ Ketogēnā	☐ Paleo	☐ Bez cukura

Alerģijas		
☐ Rieksti	☐ Piens	☐ Olas
☐ Soja	☐ Gliemenes	☐ Kvieši
☐ Zivis	☐ Sezama sēklas	☐ Selerijas

38.att. Alerģijas un diētas ierobežojumi

Administrators funkcionalitāte

Ja lietotājs ir veiksmīgi pierēģistrējies, kā administrators, navigācijas joslā parādīsies jauna sadaļa ar zobrata ikonu (skat. 39. att.).



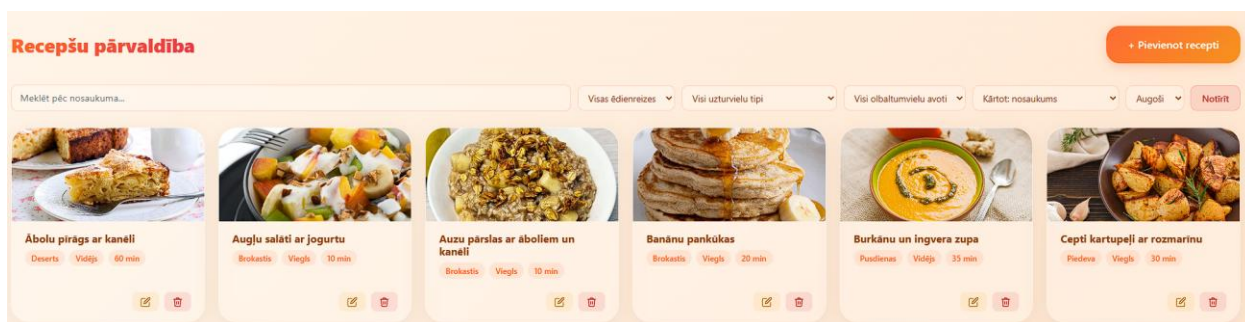
39.att. Administrators navigācijas josla

Nospiežot uz šīs ikonas parādīsies sadaļas “Receptes”, “Lietotāji” un “Sūtīt e-pastu” (skat. 40. att.).



40.att. Administratora iespējas

Sadaļā “Receptes”, administrators redz visas receptes, kas ir saglabātas datubāzē. Administratoram ir iespēja receptes pievienot/rediģēt/dzēst (skat. 41. att.).



41.att. Receptu pārvaldība

Lai pievienotu recepti, administratoram ir jāpievieno nosaukums, apraksts, pagatavošanas laiks, grūtības pakāpe, ēdienreize, uzturvielu tips, diētas tips, olbaltumvielu avots, nepieciešamās sastāvdaļas, pagatavošanas soļi un bilde/s (skat. 42. att.).

42.att. Receptes pievienošana

Sadaļā “Lietotāji”, administratoram ir iespēja pārvaldīt vai pievienot sistēmas lietotājus. Ir iespēja dzēst lietotāju vai mainīt tā lomu (skat. 43. att.).

Vārds	E-pasts	Loma	Reģistrācija	Darbības
Aleksis	ak00265@rvt.lv	Administrators	2.03.2026.	—
Aleksis	rukis.rukens@gmail.com	Lietotājs	22.03.2026.	🗑️
Aleksis Kahanovskis	aleksis.kahanovskis@gmail.com	Lietotājs	12.04.2026.	🗑️

43.att. Lietotāju pārvaldība

Lai pievienotu lietotāju, administratoram ir jāaizpilda lietotāja vārds, e-pasts, parole un loma (skat. 44. att.).

44.att. Lietotāja pievienošana

Sadaļā “Sūtīt e-pastu”, administratoram ir iespēja izvēlēties konkrētus e-pasta adresātus vai sūtīt e-pastu visiem sistēmas lietotājiem. E-pastā jānorāda temats un ziņojums (skat. 45. att.).

45.att. E-pasta sūtīšana

7. PROGRAMMATŪRAS LIETOJAMĪBAS TESTĒŠANA

Recepšu meklēšanas un filtrēšanas tests (skat. 7.1. tabulu).

7.1.tabula

Lietotāja loma:	jebkurš lietotājs (reģistrēts vai neregistrēts)	
Prasības identifikators :	RM 2.2.1.	
Prasības apraksts		
Jānodrošina, ka lietotājs var meklēt un filtrēt receptes pēc dažādiem kritērijiem, kā arī apskatīt detalizētu receptes informāciju..		
<i>Ievaddati/situācijas apraksts</i>	<i>Sagaidāmais rezultāts</i>	<i>Statuss</i>
1.Lietotājs meklēšanas laukā ievada atslēgvārdu "vista" un izvēlas grūtības pakāpi "viegls".	Tiek parādītas tikai tās receptes, kas satur vārdu "vista" un ir ar vieglu grūtības pakāpi.	pareizi
2.Lietotājs ievada meklēšanas kritērijus, kuriem neatbilst neviena recepte.	Tiek parādīts paziņojums, ka neviena recepte nav atrasta.	pareizi
3.Lietotājs uzklikšķina uz receptes no saraksta.	Tiek attēlota detalizēta receptes informācija - apraksts, sastāvdaļas, pagatavošanas soļi un laiks.	pareizi

Recepšu ģenerēšanas tests (skat. 7.2. tabulu).

7.2.tabula

Lietotāja loma:	jebkurš lietotājs (reģistrēts vai neregistrēts)	
Prasības identifikators :	RG 2.2.2	
Prasības apraksts		
Jānodrošina, ka sistēma ģenerē receptes latviešu valodā, balstoties uz lietotāja norādītajām sastāvdaļām un ierobežojumiem.		
Ievaddati/situācijas apraksts	Sagaidāmais rezultāts	Statuss
1.Lietotājs ievada sastāvdaļas "olas, burkāni" un nospiež ģenerēšanas pogu.	Sistēma parāda ģenerētu recepti ar nosaukumu, aprakstu, sastāvdaļām un pagatavošanas soļiem.	pareizi
2.Lietotājs mēģina ģenerēt recepti, neievadot nevienu sastāvdaļu.	Sistēma parāda kļūdas paziņojumu, ka jānorāda vismaz 3 sastāvdaļas.	pareizi
3.Receptes ģenerēšanas laikā rodas tehniska kļūme.	Sistēma parāda paziņojumu, ka recepti neizdevās ģenerēt.	pareizi

Recepšu pievienošanas tests (skat. 7.3. tabulu).

7.3.tabula

Lietotāja loma:	lietotājs ar administratora tiesībām	
Prasības identifikators :	RP 2.2.3	
Prasības apraksts		
Jānodrošina, ka administrators var pievienot jaunas receptes sistēmas datubāzei.		
<i>Ievaddati/situācijas apraksts</i>	<i>Sagaidāmais rezultāts</i>	<i>Statuss</i>
1.Administrators aizpilda visus obligātos laukus un pievieno recepti.	Recepte tiek veiksmīgi saglabāta un parādīts apstiprinājuma paziņojums.	pareizi

2.Administrators mēģina pievienot recepti, neizpildot visus obligātos laukus.	Sistēma parāda kļūdas paziņojumu par neizpildītajiem laukiem.	pareizi
3.Lietotājs bez administratora tiesībām mēģina pievienot recepti.	Sistēma neļauj veikt darbību un parāda paziņojumu par nepietiekamām tiesībām.	pareizi

Recepšu rediģēšanas tests (skat. 7.4. tabulu).

7.4.tabula

Lietotāja loma:	lietotājs ar administratora tiesībām	
Prasības identifikators :	RR 2.2.4	
Prasības apraksts		
Jānodrošina, ka administrators var rediģēt esošas receptes vai dzēst tās no sistēmas.		
Ievaddati/situācijas apraksts	Sagaidāmais rezultāts	Statuss
1.Administrators izmaina receptes nosaukumu un nospiež "Saglabāt izmaiņas".	Izmaiņas tiek saglabātas un parādīts paziņojums "Recepte veiksmīgi atjaunināta".	pareizi
2.Administrators izvēlas dzēst recepti un apstiprina darbību.	Recepte tiek dzēsta un administrators tiek novirzīts uz recepšu sarakstu.	pareizi
3.Lietotājs bez administratora tiesībām mēģina rediģēt recepti.	Sistēma neļauj veikt darbību un parāda paziņojumu par nepietiekamām tiesībām.	pareizi

NOBEIGUMS

Kvalifikācijas darba rezultātā ir izstrādāta daudzfunkcionāla tīmekļa lietojumprogramma "FOODYML" ēdiena recepšu meklēšanai, filtrēšanai un ģenerēšanai ar mākslīgā intelekta palīdzību. Sistēma piedāvā plašu funkcionalitāti - nodrošina recepšu meklēšanu un filtrēšanu pēc ēdienreizes, diētas tipa un proteīna avota, personalizētu recepšu filtrēšanu atbilstoši lietotāja uztura ierobežojumiem un alerģijām, recepšu ģenerēšanu ar DeepSeek mākslīgā intelekta modeli, balstoties uz norādītajām sastāvdaļām, kā arī recepšu vērtēšanu piecu zvaigžņu skalā ar komentāriem, saglabāšanu favorītos un lejupielādi PDF formātā. Administratoram ir pieejama recepšu un lietotāju pārvaldība, kā arī e-pasta nosūtīšana sistēmas lietotājiem. Sistēma tika izstrādāta, izmantojot programmēšanas valodas PHP un JavaScript, kā arī HTML un CSS priekš lietotāja saskarnes izstrādes. Servera pusē tika izmantots Laravel ietvars ar Eloquent ORM, bet lietotāja pusē - Vue 3 ietvars ar Inertia.js savienojumu. Datu bāzes pārvaldībai tika izmantota MySQL relāciju datu bāze. Recepšu ģenerēšanai tika izmantots ārējais DeepSeek API pakalpojums, bet PDF dokumentu veidošanai - DomPDF bibliotēka.

Sistēma ir paredzēta cilvēkiem, kuri gatavo ēdienu mājas apstākļos un vēlas ātri atrast vai radīt jaunas receptes latviešu valodā, ņemot vērā savus individuālos uztura ierobežojumus. Turpmāk ir iespējams attīstīt sistēmu tālāk, pievienojot papildu funkcionalitāti, kā, piemēram, sekošanu līdz uzturvielu daudzumam (kalorijas, olbaltumvielas, tauki, ogļhidrāti), nedēļas ēdienkartes plānošanu, iespēju lietotājiem iesūtīt savas receptes, kā arī mobilās lietojumprogrammas izstrādi, lai uzlabotu lietotāju saskarni un paplašinātu sistēmas pieejamību. Šādi papildinājumi ļaus vēl vairāk pielāgot sistēmu lietotāju vajadzībām un padarīt to par pilnvērtīgu ikdienas rīku ēdiena gatavošanas plānošanā.

INFORMĀCIJAS AVOTI

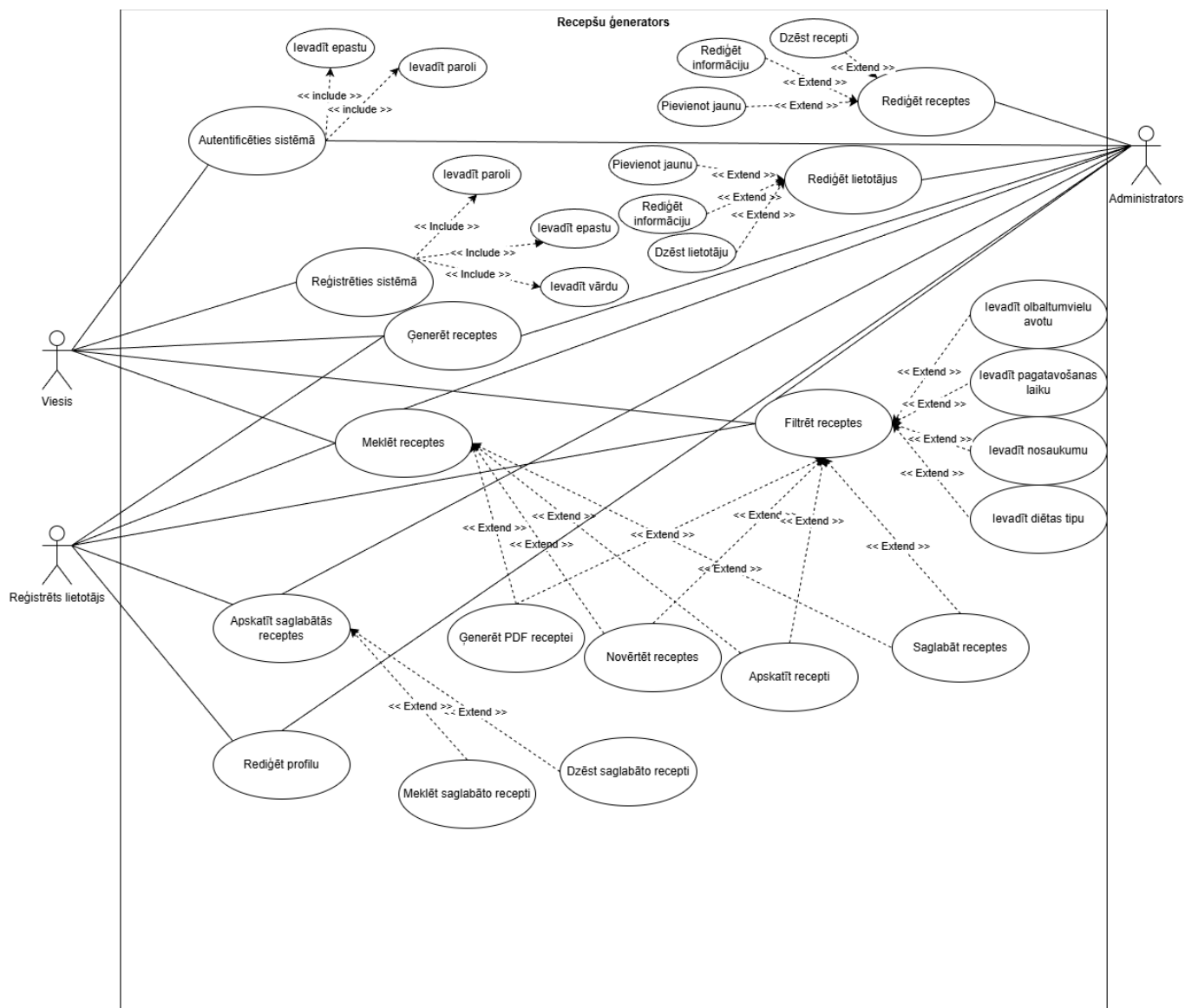
1. [ANG] Laravel Eloquent ORM. Dokumentācija - <https://laravel.com/docs/12.x/eloquent>. - (Resurss apskatīts 15.04.2026.).
2. [ANG] Laravel Sanctum - Authentication for SPAs. Dokumentācija - <https://laravel.com/docs/12.x/sanctum>. - (Resurss apskatīts 15.04.2026.).
3. [ANG] Laravel Breeze - Starter Kits. Dokumentācija - <https://laravel.com/docs/12.x/starter-kits>. - (Resurss apskatīts 15.04.2026.).
4. [ANG] Laravel Socialite - OAuth Authentication. Dokumentācija - <https://laravel.com/docs/12.x/socialite>. - (Resurss apskatīts 15.04.2026.).
5. [ANG] Asset Bundling (Vite) - Laravel. Dokumentācija - <https://laravel.com/docs/12.x/vite>. - (Resurss apskatīts 15.04.2026.).
6. [ANG] Vue.js - The Progressive JavaScript Framework. Dokumentācija - <https://vuejs.org/guide/introduction>. - (Resurss apskatīts 15.04.2026.).
7. [ANG] Inertia.js - The Modern Monolith. Dokumentācija - <https://inertiajs.com/>. - (Resurss apskatīts 15.04.2026.).
8. [ANG] Bootstrap 5 - Build fast, responsive sites. Dokumentācija - <https://getbootstrap.com/docs/5.3/getting-started/introduction/>. - (Resurss apskatīts 15.04.2026.).
9. [ANG] PHP - Hypertext Preprocessor. Dokumentācija - <https://www.php.net/docs.php>. - (Resurss apskatīts 15.04.2026.).
10. [ANG] MySQL 8.0 Reference Manual - <https://dev.mysql.com/doc/refman/8.0/en/>. - (Resurss apskatīts 15.04.2026.).
11. [ANG] DeepSeek API Documentation - Chat Completion - <https://api-docs.deepseek.com/>. - (Resurss apskatīts 15.04.2026.).
12. [ANG] barryvdh/laravel-dompdf - A DOMPDF Wrapper for Laravel. GitHub repozitorijs - <https://github.com/barryvdh/laravel-dompdf>. - (Resurss apskatīts 15.04.2026.).
13. [ANG] Axios - Promise based HTTP client. GitHub repozitorijs - <https://github.com/axios/axios>. - (Resurss apskatīts 15.04.2026.).
14. [ANG] Vite - Next Generation Frontend Tooling. Dokumentācija - <https://vite.dev/guide/>. - (Resurss apskatīts 15.04.2026.).
15. [ANG] tightenco/ziggy - Use your Laravel routes in JavaScript. GitHub repozitorijs - <https://github.com/tighten/ziggy>. - (Resurss apskatīts 15.04.2026.).
16. [ANG] Visual Studio Code - Code Editing. Redefined - <https://code.visualstudio.com/docs>. - (Resurss apskatīts 15.04.2026.).

17. [ANG] DBeaver - Universal Database Tool. Dokumentācija - <https://dbeaver.com/docs/>. -
(Resurss apskatīts 15.04.2026.).

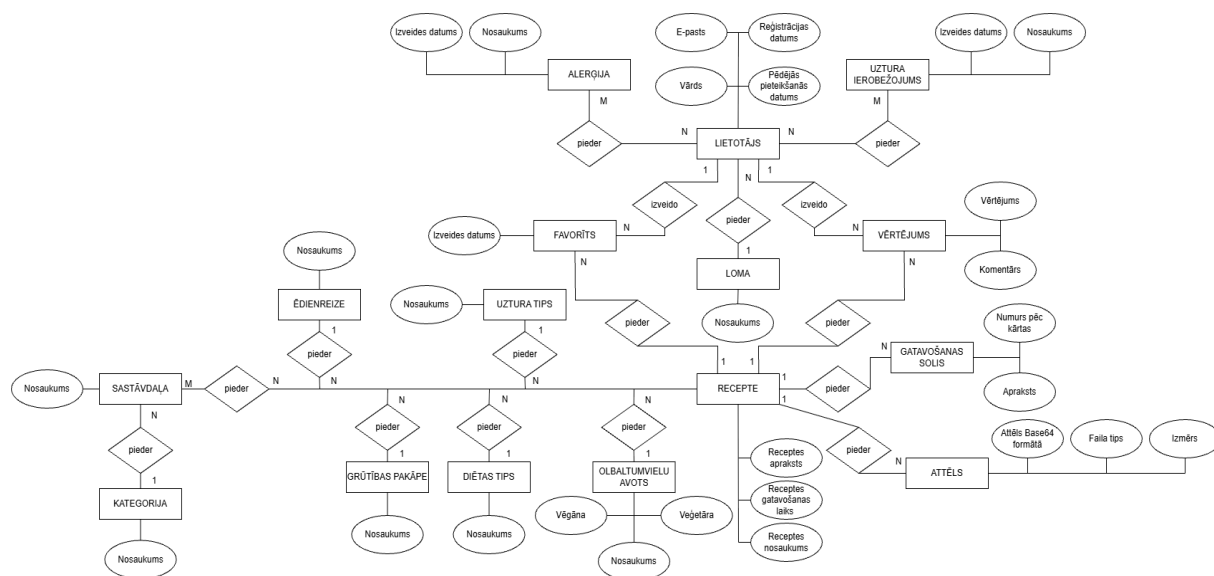
PIELIKUMI

1.pielikums

Lietojumgadījumu diagramma



ER diagramma



Programmas pirmkods

<?php

```

namespace App\Http\Controllers\Admin;

use App\Http\Controllers\Controller;
use App\Mail\AdminBroadcast;
use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Mail;
use Inertia\Inertia;

class AdminMailController extends Controller // Lapa, kur admins var sūtīt
e-pastus lietotājiem
{
    public function index()
    {
        // Paņemam visus, kas apstiprinājuši savu e-pastu
        $users = User::whereNotNull('email_verified_at')
            ->orderBy('vars')
            ->get(['id', 'vars', 'email']);

        return Inertia::render('Admin/Email', [
            'recipients' => $users,
        ]);
    }

    public function send(Request $request)
    {
        // Pārliedzināties, ka viss ir aizpildīts kā nākas
        $data = $request->validate([
            'subject' => 'required|string|max:255',
            'body' => 'required|string|max:5000',
            'recipient_ids' => 'required|array|min:1',
            'recipient_ids.*' => 'integer|exists:users,id',
        ]);

        // Nedaudz drošībai

```

```

        $users = User::whereIn('id', $data['recipient_ids'])
            ->whereNotNull('email_verified_at')
            ->get();

        // Liekam vēstules rindā, lai netraucētu galveno plūsmu
        foreach ($users as $user) {
            Mail::to($user->email)
                ->queue(new AdminBroadcast($data['subject'],
                    $data['body']));
        }

        return redirect()->route('admin.email')
            ->with('status', 'email-sent')
            ->with('sent_count', $users->count());
    }
}

```

<?php

```

namespace App\Http\Controllers\Admin;

```

```

use App\Http\Controllers\Controller;
use App\Models\DietType;
use App\Models\DifficultyLevel;
use App\Models\Image;
use App\Models\Ingredient;
use App\Models\MealTime;
use App\Models\NutritionType;
use App\Models\ProteinSource;
use App\Models\Recipe;
use App\Models\Unit;
use Illuminate\Http\Request;
use Inertia\Inertia;

```

```

class AdminRecipeController extends Controller // Šeit notiek visa recepšu
administrēšana
{
    public function index(Request $request)
    {

```



```

// Sākumā paņemam receptes kopā ar visu saistīto, lai pēc tam
neslogotu DB ar n+1
$query = Recipe::with([
    'difficultyLevel',
    'mealTime',
    'nutritionType',
    'dietType',
    'proteinSource',
    'ingredients',
    'instructions' => fn($q) => $q->orderBy('step_number'),
    'images',
]);

// Ja lietotājs kaut ko meklē vai filtrē ņemam to vērā
if ($request->filled('search')) {
    $query->where('name', 'like', '%' . $request->search . '%');
}

if ($request->filled('meal_time_id')) {
    $query->where('meal_time_id', $request->meal_time_id);
}

if ($request->filled('nutrition_type_id')) {
    $query->where('nutrition_type_id', $request->nutrition_type_id);
}

if ($request->filled('protein_source_id')) {
    $query->where('protein_source_id', $request->protein_source_id);
}

// Pārbaudām, vai sortēšanas lauks ir atļauts
$allowedSorts = ['name', 'cooking_time', 'created_at'];
$sortBy = in_array($request->get('sort_by'), $allowedSorts) ?
$request->get('sort_by') : 'name';
$sortDir = $request->get('sort_direction') === 'desc' ? 'desc' :
'asc';
$query->orderBy($sortBy, $sortDir);

```

```

        // Pa 12 receptēm uz lapu; withQueryString lai filtri saglabājas,
        kad pārslēdzas lapas
        $recipes = $query->paginate(12)->withQueryString();

        // Sūtām visu uz frontu - gan pašas receptes, gan filtru opcijas
        return Inertia::render('Admin/Receptes', [
            'recipes'          => $recipes,
            'difficultyLevels' => DifficultyLevel::all(),
            'mealTimes'        => MealTime::all(),
            'nutritionTypes'   => NutritionType::all(),
            'dietTypes'        => DietType::all(),
            'proteinSources'   => ProteinSource::all(),
            'ingredients'      => Ingredient::orderBy('name')->get(),
            'units'            => Unit::all(),
            'filters'          => $request->only(['search', 'meal_time_id',
            'nutrition_type_id', 'protein_source_id', 'sort_by', 'sort_direction']),
        ]);
    }

    public function store(Request $request)
    {
        // Pārbaudām ievadīto info pirms saglabāšanas
        $data = $request->validate([
            'name'                => 'required|string|max:255',
            'description'         => 'required|string',
            'cooking_time'        => 'required|integer|min:1',
            'difficulty_level_id' =>
            'required|exists:difficulty_levels,id',
            'meal_time_id'        => 'required|exists:meal_times,id',
            'nutrition_type_id'   =>
            'required|exists:nutrition_types,id',
            'diet_type_id'        => 'required|exists:diet_types,id',
            'protein_source_id'   =>
            'nullable|exists:protein_sources,id',
            'is_public'           => 'boolean',
            'ingredients'         => 'array',
            'ingredients.*.ingredient_id' =>
            'required|exists:ingredients,id',
            'ingredients.*.quantity' => 'required|numeric|min:0',
            'ingredients.*.unit_id' => 'required|exists:units,id',
            'instructions'        => 'array',
            'instructions.*.description' => 'required|string',
        ]);
    }

```

```

        'images' => 'nullable|array|max:5',
        'images.*' => 'image|max:5120',
    ]);

// Vispirms pati recepte
$recipe = Recipe::create([
    'name' => $data['name'],
    'description' => $data['description'],
    'cooking_time' => $data['cooking_time'],
    'difficulty_level_id' => $data['difficulty_level_id'],
    'meal_time_id' => $data['meal_time_id'],
    'nutrition_type_id' => $data['nutrition_type_id'],
    'diet_type_id' => $data['diet_type_id'],
    'protein_source_id' => $data['protein_source_id'] ?? null,
    'is_public' => $data['is_public'] ?? true,
]);

// Tagad sastāvdaļas, katrai jāzina cik daudz un kādā mērvienībā
foreach ($data['ingredients'] ?? [] as $ing) {
    $recipe->ingredients()->attach($ing['ingredient_id'], [
        'quantity' => $ing['quantity'],
        'unit_id' => $ing['unit_id'],
    ]);
}

// Soļus numurējam automātiski pēc kārtas
foreach ($data['instructions'] ?? [] as $i => $step) {
    $recipe->instructions()->create([
        'step_number' => $i + 1,
        'description' => $step['description'],
    ]);
}

// Bildes glabājam datubāzē base64 formātā
foreach ($request->file('images') ?? [] as $file) {
    $recipe->images()->create([
        'base64_data' => base64_encode($file->get()),
        'mime_type' => $file->getMimeType(),
        'original_filename' => $file->getClientOriginalName(),
        'file_size' => $file->getSize(),
    ]);
}

```

```

    }

    return redirect()->route('admin.recipes.index')
        ->with('status', 'recepte-pievienota');
}

public function update(Request $request, Recipe $recipe)
{
    // Tādi paši validācijas noteikumi kā veidojot, plus ID dzēšamajām
    bildēm
    $data = $request->validate([
        'name' => 'required|string|max:255',
        'description' => 'required|string',
        'cooking_time' => 'required|integer|min:1',
        'difficulty_level_id' =>
        'required|exists:difficulty_levels,id',
        'meal_time_id' => 'required|exists:meal_times,id',
        'nutrition_type_id' =>
        'required|exists:nutrition_types,id',
        'diet_type_id' => 'required|exists:diet_types,id',
        'protein_source_id' =>
        'nullable|exists:protein_sources,id',
        'is_public' => 'boolean',
        'ingredients' => 'array',
        'ingredients.*.ingredient_id' =>
        'required|exists:ingredients,id',
        'ingredients.*.quantity' => 'required|numeric|min:0',
        'ingredients.*.unit_id' => 'required|exists:units,id',
        'instructions' => 'array',
        'instructions.*.description' => 'required|string',
        'images' => 'nullable|array|max:5',
        'images.*' => 'image|max:5120',
        'deleted_image_ids' => 'nullable|array',
        'deleted_image_ids.*' => 'integer',
    ]);

    // Vispirms atjauninām pamata info
    $recipe->update([
        'name' => $data['name'],
        'description' => $data['description'],
        'cooking_time' => $data['cooking_time'],
        'difficulty_level_id' => $data['difficulty_level_id'],
    ]);
}

```

```

        'meal_time_id'      => $data['meal_time_id'],
        'nutrition_type_id' => $data['nutrition_type_id'],
        'diet_type_id'      => $data['diet_type_id'],
        'protein_source_id' => $data['protein_source_id'] ?? null,
        'is_public'         => $data['is_public'] ?? true,
    ));

    // Sastāvdaļas pārtaisām pilnībā, sync paņem ko atstāt un ko
    izsviest
    $syncData = [];
    foreach ($data['ingredients'] ?? [] as $ing) {
        $syncData[$ing['ingredient_id']] = [
            'quantity' => $ing['quantity'],
            'unit_id'   => $ing['unit_id'],
        ];
    }
    $recipe->ingredients()->sync($syncData);

    // Soļiem nav īpaši ko atjaunināt, vienkāršāk dzēst un izveidot no
    jauna
    $recipe->instructions()->delete();
    foreach ($data['instructions'] ?? [] as $i => $step) {
        $recipe->instructions()->create([
            'step_number' => $i + 1,
            'description' => $step['description'],
        ]);
    }

    // Ja lietotājs grib kādas bildes izņemt
    if (!empty($data['deleted_image_ids'])) {
        $recipe->images()->whereIn('id', $data['deleted_image_ids'])->
        >delete();
    }

    // Un pievienojam jaunās, ja tādas ir
    foreach ($request->file('images') ?? [] as $file) {
        $recipe->images()->create([
            'base64_data'      => base64_encode($file->get()),
            'mime_type'        => $file->getMimeType(),
            'original_filename' => $file->getClientOriginalName(),
            'file_size'        => $file->getSize(),
        ]);
    }

```

```

        ]);
    }

    return redirect()->route('admin.recipes.index')
        ->with('status', 'recepte-atjauninata');
}

public function destroy(Recipe $recipe)
{
    // dzēš no DB
    $recipe->delete();

    return redirect()->route('admin.recipes.index')
        ->with('status', 'recepte-dzesta');
}
}

<?php

namespace App\Http\Controllers\Admin;

use App\Http\Controllers\Controller;
use App\Models\Role;
use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;
use Inertia\Inertia;

class AdminUserController extends Controller // Lietotāju pārvaldība
    administratora panelī
{
    public function index()
    {
        // Visi lietotāji ar savām lomām, sakārtoti alfabēta secībā
        return Inertia::render('Admin/Lietotaji', [
            'users' => User::with('role')->orderBy('vars')->get(),
            'roles' => Role::all(),
        ]);
    }
}

```

```

}

public function store(Request $request)
{
    // Pārbaudām, vai admins ir aizpildījis visus laukus
    $data = $request->validate([
        'vars' => 'required|string|max:255',
        'email' => 'required|email|unique:users,email',
        'password' => 'required|string|min:8|confirmed',
        'password_confirmation' => 'required',
        'role_id' => 'required|exists:roles,id',
    ]);

    // Veidojam kontu paroli šifrējam ar Hash, lai DB nebūtu plain
    teksta veidā
    $user = User::create([
        'vars' => $data['vars'],
        'email' => $data['email'],
        'password' => Hash::make($data['password']),
        'role_id' => $data['role_id'],
        'registrācijas_datums' => now()->toDateString(),
        'pedeja_pieteikšanās' => now(),
    ]);

    // Tā kā admins to izveido, e-pastu uzreiz uzskatām par verificētu
    $user->markEmailAsVerified();

    return redirect()->route('admin.users.index')
        ->with('status', 'lietotajs-pievienots');
}

public function updateRole(Request $request, User $user)
{
    // Lomas maiņa to var izdarīt tikai admins
    $data = $request->validate([
        'role_id' => 'required|exists:roles,id',
    ]);

    $user->update(['role_id' => $data['role_id']]);
}

```

```

        return response()->json(['message' => 'Loma atjaunināta.']);
    }

    public function destroy(Request $request, User $user)
    {
        // Drošības pasākums
        if ($user->id === $request->user()->id) {
            abort(403, 'Nevar dzēst savu kontu.');
```

}

 // Tagad var dzēst
 \$user->delete();

 return redirect()->route('admin.users.index')
 ->with('status', 'lietotajs-dzests');
 }
}

```

<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;

class ConfigController extends Controller // Endpoint, kur frontends paņem
savas konfigurācijas vērtības
{
    public function getAppConfig()
    {
        // Visi tie izvēlnes elementi un vērtības, ko izmanto frontends
        centralizēti vienā vietā
        return response()->json([
            'portionSizes' => [
                ['value' => 1, 'label' => '1 porcija'],
                ['value' => 2, 'label' => '2 porcijas'],
            ]
        ]);
    }
}

```



```

        ['value' => 3, 'label' => '3 porcijas'],
        ['value' => 4, 'label' => '4 porcijas'],
        ['value' => 5, 'label' => '5 porcijas'],
    ],
    'sortOptions' => [
        ['value' => 'rating', 'label' => 'Vērtējuma'],
        ['value' => 'difficulty', 'label' => 'Grūtības pakāpes'],
        ['value' => 'cooking_time', 'label' => 'Gatavošanas
laika'],
    ],
    'sortDirections' => [
        ['value' => 'asc', 'label' => 'Augoši (A-Z)'],
        ['value' => 'desc', 'label' => 'Dilstoši (Z-A)'],
    ],
    'filterLabels' => [
        'allMealTimes' => 'Visas ēdienreizes',
        'allNutritionTypes' => 'Visi uztura veidi',
        'allProteinSources' => 'Visi olbaltumvielu avoti',
        'clearFilters' => 'Notīrīt filtrus',
        'sortBy' => 'Kārtot pēc...',
        'searchPlaceholder' => 'Meklēt recepti...',
    ],
    ]);
}
}

```

<?php

```
namespace App\Http\Controllers\Api;
```

```

use App\Http\Controllers\Controller;
use App\Services\AIRecipeGeneratorService;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;

```

```

// Šeit nokļūst pieprasījumi, kad lietotājs grib, lai AI izdomā recepti
class DeepSeekRecipeController extends Controller
{
    private AIRecipeGeneratorService $aiService;

```

```

public function __construct(AIRecipeGeneratorService $aiService)
{
    // Servisu Laravel injecto pats tā ērtāk testēt
    $this->aiService = $aiService;
}

public function generateRecipe(Request $request)
{
    try {
        // Sastāvdaļas obligātas, preferences pēc vēlēšanās
        $request->validate([
            'ingredients' => 'required|string',
            'use_preferences' => 'boolean',
        ]);

        $options = [];

        // Ja lietotājs ielogojies un grib, lai AI ņem vērā viņa diētas
        vai alerģijas
        if (auth()->check() && $request->boolean('use_preferences',
true)) {
            $user = auth()->user()->load(['dietaryRestrictions',
'allergies']);

            // ja ir diētas, paņemam tās
            if ($user->dietaryRestrictions->isNotEmpty()) {
                $options['dietary_restrictions'] = $user-
>dietaryRestrictions->pluck('name')->toArray();
            }

            // Tāpat ar alerģijām, lai AI nesabojā
            if ($user->allergies->isNotEmpty()) {
                $options['allergies'] = $user->allergies-
>pluck('name')->toArray();
            }
        }

        // Pasaucam servisu un gaidām atbildi
    }
}

```

```

        $result = $this->aiService->generateRecipe($request-
>ingredients, $options);

```

```

// Ja kaut kas nogāja greizi - pasakām lietotājam
if (!$result['success']) {
    return response()->json([
        'error' => 'Recipe generation failed',
        'message' => $result['error'] ?? 'Unknown error'
    ], 500);
}

```

```

// Visi laimīgi, sūtām atpakaļ recepti
return response()->json([
    'recipe' => $result['recipe']
]);

```

```

} catch (\Throwable $e) {
    // Kaut kas pavisam slikts, rakstam logā un atbildam ar kļūdu
    Log::error('Recipe generation error', [
        'message' => $e->getMessage(),
        'trace' => $e->getTraceAsString()
    ]);
}

```

```

return response()->json([
    'error' => 'Internal server error',
    'message' => $e->getMessage()
], 500);
}

```

```

}
}

```

```

<?php

```

```

namespace App\Services;

```

```

use Illuminate\Support\Facades\Http;
use Illuminate\Support\Facades\Log;

```

```

// Tilts starp programmu un DeepSeek AI
class AIRecipeGeneratorService
{
    private string $apiKey;
    private string $apiUrl = 'https://api.deepseek.com/chat/completions';
    private string $model = 'deepseek-chat';
    private float $temperature = 0.7; // 0 - vienmēr tādu pašu atbildi, 1
- katrreiz savādāku. 0.7 ir labs vidusceļš

    public function __construct()
    {
        // API atslēga jāpaņem no .env (services.deepseek.api_key)
        $this->apiKey = config('services.deepseek.api_key',
env('DEEPSEEK_API_KEY'));

        // Bez atslēgas neko nevarēsim izdarīt - labāk, lai uzreiz izkrīt,
nekā vēlāk
        if (!$this->apiKey) {
            throw new \RuntimeException('DeepSeek API key is not
configured');
        }
    }

    public function generateRecipe(string $ingredients, array $options =
[]): array
    {
        // Sagatavojam tekstu, ko sūtīsim AI
        $prompt = $this->buildPrompt($ingredients, $options);

        try {
            // Aiziet pie DeepSeek
            $response = $this->callAPI($prompt);

            // AI atbild JSON struktūrā, mums vajag tikai pašu tekstu
            return [
                'success' => true,
                'recipe' => $response['choices'][0]['message']['content']
?? '',
            ];
        }
    }
}

```

```

    } catch (\Exception $e) {
        // Logā ieraksts par kļūdu, lietotājam atgriežam neveiksmes
        paziņojumu
        Log::error('Recipe generation error', [
            'message' => $e->getMessage(),
            'trace'    => $e->getTraceAsString()
        ]);

        return [
            'success' => false,
            'error'    => $e->getMessage(),
        ];
    }
}

```

```

private function buildPrompt(string $ingredients, array $options):
string
{
    // Šis ir galvenais teksts, ko AI redzēs - jo precīzāks formāts, jo
    vieglāk pēc tam parsēt
    $basePrompt = "Izveido vienu īsu recepti no šiem produktiem (nav
    obligāti jāizmanto visas sastāvdaļas): {$ingredients} (Nav pieejama neviena
    cita sastāvdaļa).

```

Formāts (OBLIGĀTI JĀIEVĒRO):

Nosaukums:

[receptes nosaukums]

Sastāvdaļas:

- [sastāvdaļa ar daudzumu]
- [sastāvdaļa ar daudzumu]

Pagatavošana:

1. [pirmais solis]
2. [otrais solis]
3. [trešais solis]

SVARĪGI: Katrs pagatavošanas solis JĀBŪT atsevišķā rindā ar numuru. Neraksti visus soļus vienā rindā. Neizmanto markdown formātus.";

```
// Pievienojam diētas, ja tādas ir
if (!empty($options['dietary_restrictions'])) {
    $basePrompt .= " Ievēro šādus ierobežojumus: " . implode(', ',
$options['dietary_restrictions']) . ".";
}

// Un alerģijas
if (!empty($options['allergies'])) {
    $basePrompt .= " Izvairīties no: " . implode(', ',
$options['allergies']) . ".";
}

return $basePrompt;
}

private function callAPI(string $prompt): array
{
    // HTTP pieprasījums - 60s timeout, 2x atkārtot
    $response = Http::timeout(60)
        ->retry(2, 1000)
        ->withHeaders([
            'Authorization' => 'Bearer ' . $this->apiKey,
            'Content-Type' => 'application/json',
        ])
        ->post($this->apiUrl, [
            'model' => $this->model,
            'messages' => [
                [
                    // Ar šo AI "noskaņojas" kā pavārs, kas atbild
latviski
                    'role' => 'system',
                    'content' => 'Tu esi pavārs. Atbildi tikai
latviski.'
                ],
                [
                    // Pati prasība par recepti
                    'role' => 'user',
                    'content' => $prompt
                ]
            ]
        ]
    );
}
```

```

        ],
    ],
    'temperature' => $this->temperature,
    'max_tokens' => 1500, // 1500 tokenu vajadzētu būt pilnīgi
pietiekoši
]);

```

```

// Ja DeepSeek atgrieza kļūdu - metam exception, lai augstāk var to
apstrādāt

```

```

    if ($response->failed()) {
        throw new \RuntimeException(
            'DeepSeek API request failed: ' . $response->status() . '
- ' . $response->body()
        );
    }

```

```

// JSON automātiski iegūstam kā masīvu
return $response->json();
}

```

```

// Setteris, ja kādreiz vajag mainīt modeli no ārpusēs

```

```

public function setModel(string $model): self
{
    $this->model = $model;
    return $this;
}

```

```

// Tāpat ar temperatūru (radošumu)

```

```

public function setTemperature(float $temperature): self
{
    $this->temperature = $temperature;
    return $this;
}

```

```

}

```

```

<?php

```

```

namespace App\Http\Controllers\Api;

```

```

use App\Models\Ingredient;
use App\Http\Controllers\Controller;

class IngredientController extends Controller // Vienkāršs API endpoints
sastāvdaļu sarakstam
{
    public function index()
    {
        // Visas sastāvdaļas, sagrupētas pa kategorijām (gaļa, dārzeņi utt.)
        frontendā tā ērtāk parādīt
        $ingredients = Ingredient::with('category')->get()->groupBy(fn($i)
=> $i->category->name);
        return response()->json($ingredients);
    }
}

```

<?php

```

namespace App\Http\Controllers\Api;

```

```

use App\Http\Controllers\Controller;
use App\Http\Resources\RecipeResource;
use App\Models\Image;
use App\Services\RecipeService;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;

```

```

class RecipeController extends Controller // Galvenais recepšu API
kontrolieris
{
    private RecipeService $recipeService;

    public function __construct(RecipeService $recipeService)
    {
        $this->recipeService = $recipeService;
    }
}

```



```

public function index(Request $request)
{
    try {
        // Visi iespējamie filtru parametri no URL
        $filters = $request->only([
            'search',
            'meal_time',
            'nutrition',
            'protein_source',
            'diet_type',
            'difficulty',
            'filter_by_preferences',
            'favorites_only'
        ]);
        $sortBy          = $request->input('sort_by');
        $sortDirection = $request->input('sort_direction', 'asc');
        $perPage         = $request->input('per_page', 10);

        // Pārējo darbu dara serviss
        $recipes = $this->recipeService->getRecipes($filters, $sortBy,
        $sortDirection, $perPage);

        return RecipeResource::collection($recipes);
    } catch (\Exception $e) {
        Log::error('Recipe fetch error: ' . $e->getMessage());
        return response()->json(['error' => 'Server error'], 500);
    }
}

public function getFilters()
{
    try {
        // Apgādājam frontu ar filtru opcijām
        return response()->json($this->recipeService->getFilters());
    } catch (\Exception $e) {
        Log::error('Filter fetch error: ' . $e->getMessage());
        return response()->json(['error' => 'Server error'], 500);
    }
}

public function show($id)

```

```

{
  try {
    // Vienas receptes detaļas
    $recipe = $this->recipeService->getRecipeById($id);

    if (!$recipe) {
      return response()->json(['message' => 'Recepte nav
atrasta!'], 404);
    }

    return new RecipeResource($recipe);
  } catch (\Exception $e) {
    Log::error('Recipe detail error: ' . $e->getMessage());
    return response()->json(['error' => 'Server error'], 500);
  }
}

public function serveImage($id)
{
  // Bilde glabājas DB kā base64
  $image = Image::findOrFail($id);

  // Atgriežam neapstrādātos baitus ar kešošanas headeriem
  return response($image->base64_data_raw, 200)
    ->header('Content-Type', $image->mime_type)
    ->header('Cache-Control', 'public, max-age=31536000,
immutable');
}

public function downloadPdf($id)
{
  try {
    // Receptes PDF lejupielādei
    $pdf = $this->recipeService->generateRecipePdf($id);

    // Kodējam base64 JS tad var izveidot lejupielādes saiti
    return response()->json([
      'pdf' => base64_encode($pdf->output()),
      'filename' => "recepte-{$id}.pdf",
    ]);
  }
}

```

```

    });
} catch (\Exception $e) {
    Log::error('PDF generation error: ' . $e->getMessage());
    return response()->json(['error' => 'Kļūda ģenerējot PDF'],
500);
}
}
}
}

```

<?php

```

namespace App\Http\Controllers\Api;

```

```

use App\Http\Controllers\Controller;
use App\Models\Recipe;
use App\Models\Unit;
use Barryvdh\DomPDF\Facade\Pdf;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Log;

```

```

class ShoppingListController extends Controller // Ģenerē iepirkumu
sarakstus no izvēlētajām receptēm

```

```

{
    public function generate(Request $request)
    {
        $request->validate([
            'recipes' => 'required|array|min:1|max:20',
            'recipes.*.recipe_id' => 'required|integer|exists:recipes,id',
            'recipes.*.portions' => 'required|numeric|min:0.5|max:100',
        ]);

        // Apvienojam visu vajadzīgo vienā sarakstā
        $list = $this->buildList($request->input('recipes'));

        return response()->json(['list' => $list]);
    }
}

```

```

    public function downloadPdf(Request $request)

```

```

{
    $request->validate([
        'recipes' => 'required|array|min:1|max:20',
        'recipes.*.recipe_id' => 'required|integer|exists:recipes,id',
        'recipes.*.portions' => 'required|numeric|min:0.5|max:100',
    ]);

    try {
        $recipeItems = $request->input('recipes');
        $list         = $this->buildList($recipeItems);

        // PDFā vēlamies redzēt arī kuras receptes ir iekļautas
        $recipeIds    = collect($recipeItems)->pluck('recipe_id');
        $recipes      = Recipe::whereIn('id', $recipeIds)->get(['id',
'name'])->keyBy('id');

        $recipeNames = collect($recipeItems)->map(fn($item) => [
            'name' => $recipes[$item['recipe_id']]?->name ??
'Nezināma recepte',
            'portions' => $item['portions'],
        ]);

        // Renderējam no Blade šablona
        $pdf = Pdf::loadView('pdfs.shopping_list', [
            'list' => $list,
            'recipeNames' => $recipeNames,
            'generatedAt' => now()->format('d.m.Y H:i'),
        ]);

        // kodējam base64 un atgriežam uz UI, js to uzreiz lejupielādē
        return response()->json([
            'pdf' => base64_encode($pdf->output()),
            'filename' => 'iepirkumu-saraksts.pdf',
        ]);
    } catch (\Exception $e) {
        Log::error('Shopping list PDF generation error: ' . $e-
>getMessage());
        return response()->json(['error' => 'Kļūda ģenerējot PDF'],
500);
    }
}

```

```

    }

    private function buildList(array $recipeItems): array
    {
        $recipeIds    = collect($recipeItems)->pluck('recipe_id');
        $portionsMap   = collect($recipeItems)->keyBy('recipe_id')->map(fn($i) => floatval($i['portions']));

        // Visas receptes
        $recipes = Recipe::whereIn('id', $recipeIds)
            ->with(['ingredients' => fn($q) => $q->with('category')])
            ->get();

        $combined = [];

        // Reizinām ar porciju skaitu un summējam, ja sastāvdaļa atkārtojas
        foreach ($recipes as $recipe) {
            $portions = $portionsMap[$recipe->id] ?? 1;

            foreach ($recipe->ingredients as $ingredient) {
                $unitId = $ingredient->pivot->unit_id;
                $key     = $ingredient->id . '_' . $unitId;

                if (isset($combined[$key])) {
                    $combined[$key]['quantity'] += floatval($ingredient->pivot->quantity) * $portions;
                } else {
                    $combined[$key] = [
                        'name'      => $ingredient->name,
                        'category'  => $ingredient->category?->name ??
'Cits',
                        'quantity' => floatval($ingredient->pivot->quantity) * $portions,
                        'unit_id'  => $unitId,
                        'unit'     => '',
                    ];
                }
            }
        }
    }
}

```

```

// Tagad piesaistām mērvienību nosaukumus
$unitIds      =      collect($combined)->pluck('unit_id')->unique()-
>filter();
$units        = Unit::whereIn('id', $unitIds)->pluck('name', 'id');

foreach ($combined as &$item) {
    $item['unit'] = $units[$item['unit_id']] ?? '';
    unset($item['unit_id']);
}
unset($item);

// Beigās sagrupējam pa kategorijām
return collect($combined)
    ->groupBy('category')
    ->map(fn($items, $category) => [
        'category'    => $category,
        'ingredients' => $items
        ->map(fn($i) => [
            'name'      => $i['name'],
            'quantity' => round($i['quantity'], 2),
            'unit'      => $i['unit'],
        ])
        ->sortBy('name')
        ->values(),
    ])
    ->sortKeys()
    ->values()
    ->toArray();
}
}

```

<?php

```
namespace App\Http\Controllers\Auth;
```

```

use App\Http\Controllers\Controller;
use App\Models\Role;
use App\Models\User;

```

```

use Illuminate\Support\Facades\Auth;
use Laravel\Socialite\Facades\Socialite;

class SocialAuthController extends Controller // Pieslēgšanās caur Google
kontu
{
    public function redirect()
    {
        // Aizsūtām uz Google, lai cilvēks tur autorizējas
        return Socialite::driver('google')->redirect();
    }

    public function callback()
    {
        // Atpakaļ no Google ar lietotāja datiem
        $googleUser = Socialite::driver('google')->user();

        // Vai mums jau ir tāds e-pasts datubāzē?
        $user = User::where('email', $googleUser->getEmail())->first();

        if ($user) {
            // Ja jā, bet vēl nav saistīts ar Google, sasienam abus kopā
            if (!$user->social_id) {
                $user->update([
                    'social_provider' => 'google',
                    'social_id'       => $googleUser->getId(),
                    'email_verified_at' => $user->email_verified_at ??
now(),
                ]);
            }
        } else {
            // Pavisam jauns lietotājs - veidojam no nulles ar standarta
lomu

            $role = Role::where('name', 'Lietotājs')->first();

            $user = User::create([
                'vars' => $googleUser->getName(),
                'email' => $googleUser->getEmail(),
                'password' => null,
                'social_provider' => 'google',
            ]);
        }
    }
}

```

```

        'social_id'          => $googleUser->getId(),
        'role_id'           => $role->id,
        'registrācijas_datums' => today(),
        'email_verified_at'  => now(),
    ]);
}

// Iekšā un mājup
Auth::login($user);

return redirect()->route('home');
}
}

<?php

namespace App\Http\Controllers;

use App\Models\Favorite;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Log;

class FavoriteController extends Controller // Saglabāto receptu pārvaldība
{
    public function store(Request $request)
    {
        try {
            // Vajadzīgs tikai recipe_id
            $data = $request->validate([
                'recipe_id' => 'required|exists:recipes,id',
            ]);

            $userId = Auth::id();

            // Pārbaudām, lai nedublētos - viena recepte favorītos var būt
            tikai vienreiz

```



```

    $exists = Favorite::where('user_id', $userId)
        ->where('recipe_id', $data['recipe_id'])
        ->exists();

    if ($exists) {
        return response()->json(['error' => 'Recipe is already
saved'], 409);
    }

    // Aiziet favoritos
    Favorite::create([
        'user_id' => $userId,
        'recipe_id' => $data['recipe_id'],
    ]);

    return response()->json(['saved' => true, 'has_favorites' =>
true], 201);

} catch (\Exception $e) {
    Log::error('Favorite store error: ' . $e->getMessage());
    return response()->json([
        'error' => 'Server error',
        'details' => $e->getMessage()
    ], 500);
}
}

public function destroy($recipeId)
{
    try {
        $userId = Auth::id();

        // Atrodam ierakstu
        $favorite = Favorite::where('user_id', $userId)
            ->where('recipe_id', $recipeId)
            ->first();

        if (!$favorite) {

```

```

        return response()->json(['error' => 'Favorite not found'],
404);
    }

    // Un dzēšam
    $favorite->delete();

    // lai parādītu/slēptu sarakstu
    $hasFavorites = Favorite::where('user_id', $userId)->exists();

    return response()->json(['saved' => false, 'has_favorites' =>
$hasFavorites], 200);

    } catch (\Exception $e) {
        Log::error('Favorite destroy error: ' . $e->getMessage());
        return response()->json([
            'error' => 'Server error',
            'details' => $e->getMessage()
        ], 500);
    }
}
}
}

```

<?php

```

namespace App\Http\Controllers;

use App\Models\DietaryRestriction;
use App\Models>Allergy;
use App\Http\Requests\ProfileUpdateRequest;
use Illuminate\Contracts\Auth\MustVerifyEmail;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Redirect;
use Inertia\Inertia;
use Inertia\Response;

```

```

class ProfileController extends Controller // Lietotāja profila iestatījumi
{
  public function edit(Request $request): Response
  {
    // Paņemam lietotāju kopā ar diētām un alerģijām
    $user = $request->user()->load([
      'dietaryRestrictions.restrictedIngredients',
      'allergies.allergicIngredients'
    ]);

    // Sūtām uz frontu visu, kas vajadzīgs profila lapai
    return Inertia::render('Profile/Edit', [
      'mustVerifyEmail' => $user instanceof MustVerifyEmail,
      'status'          => session('status'),
      'user'            => [
        'vars'          => $user->vars,
        'email'         => $user->email,
        'forbidden_ingredients'=> $user->getForbiddenIngredientIds(),
        'dietas_ierobezojumi' => $user->dietaryRestrictions->pluck('id'),
        'alerģijas'      => $user->allergies->pluck('id'),
        'is_google_user' => (bool) $user->social_provider,
      ],
      'dietas' => DietaryRestriction::all(['id', 'name']),
      'alerģijas'=> Allergy::all(['id', 'name']),
    ]);
  }

  public function update(ProfileUpdateRequest $request): RedirectResponse
  {
    $user = $request->user();

    // Atjauninām vārdu un e-pastu
    $user->fill($request->validated());

    // Ja mainīja e-pastu tas atkal jāverificē, bet frontā tas ir aizliegts
    if ($user->isDirty('email')) {
      $user->email_verified_at = null;
    }
  }
}

```

```

    }

    // Saglabājam
    $user->save();

    // Diētas un alerģijas sync sakārto tabulas
    $user->dietaryRestrictions()->sync($request-
>input('dietas_ierobejojumi', []));
    $user->allergies()->sync($request->input('alerģijas', []));

    return Redirect::route('profile.edit')->with('status', 'profils-
atjauninats');
}

public function destroy(Request $request): RedirectResponse
{
    $user = $request->user();

    // Parastiem lietotājiem pirms dzēšanas jāievada parole. Google
    lietotājiem tās vienkārši nav
    if (!$user->social_provider) {
        $request->validate([
            'password' => ['required', 'current_password'],
        ]);
    }

    // Iziet, dzēst kontu, sesiju kill, jaunu tokenu un atgriezt uz home
    Auth::logout();
    $user->delete();
    $request->session()->invalidate();
    $request->session()->regenerateToken();

    return Redirect::to('/');
}
}

```

<?php

```

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use App\Models\Rating;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;

class RatingController extends Controller // Receptu vērtēšana un komentāri
{
    public function store(Request $request)
    {
        // Vērtējumam jābūt no 1 līdz 5, komentārs neobligāts
        $data = $request->validate([
            'recipe_id' => 'required|exists:recipes,id',
            'rating'     => 'required|integer|min:1|max:5',
            'comment'    => 'nullable|string',
        ]);

        // updateOrCreate - ja jau bija novērtējis, atjaunina; ja nē,
        // izveido jaunu
        $rating = Rating::updateOrCreate(
            ['user_id' => Auth::id(), 'recipe_id' => $data['recipe_id']],
            ['rating' => $data['rating'], 'comment' => $data['comment']] ??
            null
        );

        // Pārreķinām vidējo, lai uzreiz var parādīt jauno vērtējumu
        $average = Rating::where('recipe_id', $data['recipe_id'])-
            >avg('rating');

        return response()->json([
            'rating' => $rating,
            'average' => (float) number_format($average, 1)
        ], 201);
    }

    public function destroy(int $recipeId)

```

```

{
    // Lietotājs var atsaukt savu vērtējumu
    $deleted = Rating::where('user_id', Auth::id())
        ->where('recipe_id', $recipeId)
        ->delete();

    if (!$deleted) {
        return response()->json(['message' => 'Vērtējums nav
atrasts.'], 404);
    }

    // Bez šī vērtējuma vidējais var mainīties - pārrēķinām
    $average = Rating::where('recipe_id', $recipeId)->avg('rating');

    return response()->json([
        'average' => $average ? (float) number_format($average, 1) :
0.0
    ]);
}
}

```

<?php

```
namespace App\Http\Middleware;
```

```
use Closure;
```

```
use Illuminate\Http\Request;
```

```
class EnsureUserIsAdmin // Middleware, kas neielaiž svešus admin sadaļās
```

```
{
```

```
    public function handle(Request $request, Closure $next)
```

```
    {
```

```
        if (!auth()->check() || auth()->user()->role->name !==
```

```
'Administrators') {
```

```
            abort(403);
```

```
        }
```

```

        return $next($request);
    }
}

<?php

namespace App\Services;

use App\Models\Recipe;
use App\Models\Favorite;
use App\Models\MealTime;
use App\Models\NutritionType;
use App\Models\ProteinSource;
use App\Models\Unit;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Cache;
use Barryvdh\DomPDF\Facade\Pdf;

class RecipeService // Recepšu galvenā loģika - kontrolieri tikai sauc
servisu
{
    public function getRecipes(array $filters = [], ?string $sortBy = null,
?string $sortDirection = 'asc', int $perPage = 10)
    {
        // ielādējam visu, kas vajadzīgs
        $query = Recipe::with([
            'ingredients.category',
            'ratings',
            'favorites',
            'image',
            'difficultyLevel',
            'mealTime',
            'nutritionType',
            'dietType',
            'proteinSource',
        ])
        ->select('recipes.*')
        ->withAvg('ratings as average_rating', 'rating');

        // Filtri, ja kādi ir

```

```

$this->applyFilters($query, $filters);

// Sortēšana, ja norādīta
if ($sortBy && $sortDirection) {
    $query = $this->applySorting($query, $sortBy, $sortDirection);
}

// sadalam lapās
return $query->paginate($perPage);
}

private function applySorting($query, string $sortBy, string
$sortDirection)
{
    // match - atkarībā no sortBy izvēlamies sortēšanas loģiku
    match ($sortBy) {
        'rating',          'average_rating'          => $query-
>orderBy('average_rating', $sortDirection),
        'cooking_time'      => $query->orderBy('cooking_time',
$sortDirection),
        'difficulty'        => $query-
>leftJoin('difficulty_levels',          'difficulty_levels.id',          '=',
'recipes.difficulty_level_id')
        -
        => $query->orderByRaw("FIELD(difficulty_levels.name, 'Viegls', 'Vidējs', 'Grūts') "
. ($sortDirection === 'desc' ? 'DESC' : 'ASC')),
        default              => null,
    };

    return $query;
}

public function getRecipeById(int $id): ?Recipe
{
    // Vienas receptes detaļas
    return Recipe::with([
        'instructions',
        'ingredients.category',
        'ratings.user',
        'favorites',
    ]);
}

```



```

        'image',
        'images',
        'difficultyLevel',
        'mealTime',
        'nutritionType',
        'dietType',
        'proteinSource',
    ])
    ->withAvg('ratings as average_rating', 'rating')
    ->find($id);
}

public function getFilters(): array
{
    // Filtri reti mainās - kešojam stundu
    return Cache::remember('recipe_filters', 3600, function() {
        return [
            'mealTimes'      => MealTime::pluck('name')->values(),
            'nutritionTypes' => NutritionType::pluck('name')->values(),
            'proteinSources' => ProteinSource::pluck('name')->values(),
        ];
    });
}

public function getUserFavoriteIds(?int $userId = null): array
{
    $userId = $userId ?? Auth::id();

    // Anonīmiem lietotājiem favorītu nav
    if (!$userId) {
        return [];
    }

    // Tikai ID, vairāk šeit nekas nav vajadzīgs
    return Favorite::where('user_id', $userId)
        ->pluck('recipe_id')
        ->toArray();
}

```

```

private function applyFilters($query, array $filters): void
{
    // Meklēšana pēc nosaukuma
    if (!empty($filters['search'])) {
        $query->where('name', 'like', '%' . $filters['search'] . '%');
    }

    // Brokastis / pusdienas / vakariņas
    if (!empty($filters['meal_time'])) {
        $query->whereHas('mealTime', fn($q) => $q->where('name',
$filters['meal_time']));
    }

    // Vegānisks, veģetārs utt.
    if (!empty($filters['nutrition'])) {
        $query->whereHas('nutritionType', fn($q) => $q->where('name',
$filters['nutrition']));
    }

    // Olbaltumvielu avots
    if (!empty($filters['protein_source'])) {
        $query->whereHas('proteinSource', fn($q) => $q->where('name',
$filters['protein_source']));
    }

    // Diētas tips
    if (!empty($filters['diet_type'])) {
        $query->whereHas('dietType', fn($q) => $q->where('name',
$filters['diet_type']));
    }

    // Cik grūta recepte
    if (!empty($filters['difficulty'])) {
        $query->whereHas('difficultyLevel', fn($q) => $q-
>where('name', $filters['difficulty']));
    }
}

```

```

        // Ja lietotājs grib, lai sistēma ņem vērā viņa profila
ierobežojumus - izņemam visu, kas neder
        if (!empty($filters['filter_by_preferences']) && Auth::check()) {
            $user = Auth::user() -
>load(['dietaryRestrictions.restrictedIngredients',
'allergies.allergicIngredients']);
            $forbiddenIds = $user->getForbiddenIngredientIds();

            if (!empty($forbiddenIds)) {
                $query->whereDoesntHave('ingredients', function($q) use
($forbiddenIds) {
                    $q->whereIn('ingredients.id', $forbiddenIds);
                });
            }
        }

        // "Tikai favorīti" filtrs
        if (!empty($filters['favorites_only']) && Auth::check()) {
            $favoriteIds = $this->getUserFavoriteIds(Auth::id());
            $query->whereIn('id', $favoriteIds);
        }
    }

    public function generateRecipePdf(int $recipeId)
    {
        // Visa receptes info PDF vajadzībām
        $recipe = Recipe::with([
            'instructions',
            'ingredients.category',
            'image',
            'difficultyLevel',
            'mealTime',
            'nutritionType',
            'dietType',
            'proteinSource',
        ])
        ->withAvg('ratings as average_rating', 'rating')
        ->findOrFail($recipeId);

        // Mērvienības tāpat kešojam - tās praktiski nemainās

```

```

        $unitNames = Cache::remember('units_map', 3600, fn() =>
Unit::pluck('name', 'id'));

        // PDF no Blade šablona, A4 portrait
        $pdf = Pdf::loadView('pdfs.recipe', compact('recipe',
'unitNames'));
        $pdf->setPaper('A4', 'portrait');

        return $pdf;
    }
}

```